

ВВЕДЕНИЕ

Актуальность темы. В условиях стремительного развития информационных технологий языки программирования играют ключевую роль в разработке программного обеспечения, анализе данных и создании веб-приложений. Постоянное появление новых языков и эволюция существующих требуют систематизированного подхода к хранению и анализу информации о них. Традиционные реляционные базы данных не всегда эффективны для представления сложных взаимосвязей между языками программирования, их парадигмами, фреймворками и учебными материалами.

Семантические технологии, основанные на онтологиях и графовых моделях данных, позволяют преодолеть эти ограничения. Онтологии обеспечивают формализованное описание предметной области, устанавливая связи между сущностями и облегчая семантический поиск. Разработка семантической электронной библиотеки для представления информации о языках программирования является актуальной задачей, способствующей созданию интеллектуальных систем управления знаниями.

Цель работы — разработка прототипа веб-приложения семантической электронной библиотеки по программированию, интегрирующей данные из различных источников и обеспечивающей расширенный поиск по онтологии.

Задачи исследования:

1. Анализ подходов к созданию электронных библиотек и применению семантических технологий.
2. Изучение источников данных о языках программирования (TIOBE Index, GitHub API, Stack Overflow API) и методов их интеграции.
3. Проектирование онтологии предметной области «Языки программирования».
4. Разработка архитектуры веб-приложения и реализация модулей сбора, обработки и хранения данных.
5. Создание модуля семантического поиска и веб-интерфейса для взаимодействия с библиотекой.

Объект исследования — процессы сбора, структурирования и представления данных о языках программирования в семантических информационных системах.

Предмет исследования — методы и технологии разработки семантической электронной библиотеки на основе онтологий.

Методы исследования: анализ научной литературы, проектирование онтологий, веб-разработка, интеграция API.

Научная новизна заключается в разработке комплексной онтологии, объединяющей данные о популярности языков программирования (TIOBE), их

активности в сообществе (GitHub, Stack Overflow) и связанных учебных материалах, что позволяет осуществлять многоаспектный семантический поиск.

Практическая значимость. Разработанный прототип может быть использован в образовательных целях для изучения языков программирования, а также как основа для более сложных систем анализа технологических трендов.

Реализованные модули сбора данных демонстрируют практические подходы к интеграции разнородных источников информации.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ СЕМАНТИЧЕСКОЙ ЭЛЕКТРОННОЙ БИБЛИОТЕКИ

1.1. Электронные библиотеки: понятие, классификация, особенности

Электронная библиотека — это информационная система для хранения, организации и предоставления доступа к электронным документам через компьютерные сети. В отличие от традиционных библиотек, электронные библиотеки работают с цифровыми объектами, что устраняет географические и временные ограничения, обеспечивая круглосуточный доступ к ресурсам из любой точки мира.

Классификация электронных библиотек:

1. По масштабу и доступности:

- Локальные — ограничены конкретной организацией
- Корпоративные — объединяют ресурсы нескольких организаций
- Публичные — открыты для широкого круга пользователей

2. По типу контента:

- Специализированные — ориентированы на определенную предметную область
- Универсальные — содержат разнородные материалы
- Архивные — сохраняют исторические и культурные документы

3. По функциональности:

- Репозитории — системы хранения и базового поиска
- Библиографические системы — с развитыми каталогами и метаданными
- Семантические библиотеки — с поддержкой онтологий и расширенного поиска

Особенности современных электронных библиотек:

- Использование стандартов метаданных (Dublin Core, MARC)
- Поддержка форматов представления знаний (RDF, OWL)
- Возможности семантического поиска и навигации
- Интеграция с внешними источниками данных через API
- Адаптивные веб-интерфейсы для различных устройств

Эволюция библиотечных систем прошла путь от простых каталогов до интеллектуальных систем. Современные тенденции включают переход к облачным

технологиям, использование искусственного интеллекта для рекомендаций и внедрение блокчейна для управления правами доступа.

Выводы. Электронные библиотеки представляют собой динамично развивающуюся область информационных технологий. Их эффективность зависит от качества организации метаданных, поддержки стандартов и использования передовых технологий обработки информации, включая семантические подходы.

1.2. Семантические технологии и онтологии в библиотечных системах

Семантические технологии — это методы и инструменты, направленные на придание информации машиночитаемого смысла, что позволяет компьютерам понимать содержание данных и осуществлять интеллектуальную обработку. В библиотечных системах эти технологии открывают новые возможности для организации знаний и улучшения поисковых механизмов.

Основные компоненты семантического веба:

- 1. **RDF (Resource Description Framework)** — модель данных для представления информации в виде триплетов «субъект-предикат-объект».
- 2. **OWL (Web Ontology Language)** — язык описания онтологий, расширяющий возможности RDF, поддерживает логические выводы.
- 3. **SPARQL** — язык запросов к данным в формате RDF, аналогичен SQL для графовых структур.

Функции онтологий в библиотеках:

- Унификация терминологии
- Структурирование знаний
- Поддержка логического вывода
- Улучшение поиска

Примеры библиотечных онтологий:

- Dublin Core — стандарт метаданных
- Bibframe — описание библиографических данных
- FOAF — описание людей и социальных связей
- SKOS — представление тезаурусов и классификаций

Преимущества семантических технологий:

Преимущество	Описание
Интероперабельность	Объединение данных из разных источников
Расширенный поиск	Поддержка синонимов и контекстного поиска

Преимущество	Описание
Персонализация	Адаптация результатов под пользователя
Интеграция знаний	Создание единого информационного пространства

Принципы семантического поиска:

1. Понимание намерений пользователя
2. Расширение запросов синонимами
3. Ранжирование по семантической близости
4. Фасетная навигация

Вызовы внедрения:

- Сложность создания и поддержки онтологий
- Необходимость преобразования данных в семантические форматы
- Высокие требования к вычислительным ресурсам
- Проблемы масштабирования для больших объёмов данных

Выводы. Семантические технологии и онтологии являются мощным инструментом для создания интеллектуальных библиотечных систем. Они позволяют преодолеть ограничения традиционного поиска, обеспечивая более точный и релевантный доступ к информации. Успешное внедрение требует тщательного проектирования онтологий, соответствующих потребностям предметной области и пользователей.

1.3. Обзор источников данных для формирования библиотечных коллекций по программированию

Формирование качественной коллекции данных о языках программирования требует интеграции информации из разнородных источников, каждый из которых предоставляет уникальные метрики и аспекты оценки. В рамках исследования выбраны три ключевых источника, отражающих различные грани популярности и использования языков.

1.3.1. TIOBE Index как индикатор популярности языков программирования

TIOBE Programming Community Index — авторитетный рейтинг популярности языков программирования на основе анализа поисковых запросов в основных поисковых системах.

Ключевые характеристики:

- Периодичность обновления: ежемесячно
- Охват: более 25 языков программирования
- Метрика: процентная доля популярности от общего числа запросов

- Исторические данные: доступны с 2001 года

Преимущества:

1. Объективность — основан на поведенческих данных пользователей
2. Динамичность — позволяет отслеживать изменения популярности
3. Широкое признание — используется в академических исследованиях и индустрии

Ограничения:

- Отражает популярность в поиске, а не фактическое использование
- Возможны искажения из-за особенностей алгоритмов поисковых систем
- Не учитывает специализированные нишевые языки

1.3.2. GitHub API как источник статистики разработки

GitHub — крупнейшая платформа для хостинга исходного кода. GitHub API предоставляет доступ к статистическим данным, отражающим реальную активность разработчиков.

Доступные метрики:

- Количество репозиторий по языкам
- Количество звёзд как показатель популярности проектов
- Активность коммитов и частота обновлений

Значение для библиотеки:

1. Реальное использование — отражает практическое применение языков
2. Активность сообщества — показывает размер и активность экосистемы
3. Тренды развития — выявление растущих и угасающих технологий

Особенности работы:

- Ограничение rate limiting
- Необходимость использования персональных токенов доступа
- Поддержка пагинации для больших наборов данных

1.3.3. Stack Overflow API для анализа сообщества разработчиков

Stack Overflow — крупнейшее сообщество разработчиков для решения технических вопросов. API предоставляет доступ к статистике вопросов, тегов и активности.

Ключевые данные:

- Количество вопросов по тегам языков
- Процент принятых ответов как показатель качества обсуждений

- Распределение по сложности вопросов

Преимущества:

1. Проблемно-ориентированный подход — отражает реальные трудности разработчиков
2. Качество сообщества — процент решённых вопросов как индикатор зрелости
3. Образовательная ценность — полезно для учебных материалов

Сравнительный анализ источников:

Критерий	TIOBE Index	GitHub API	Stack Overflow API
Тип метрики	Популярность в поиске	Активность разработки	Активность сообщества
Периодичность	Ежемесячно	Реальное время	Постоянно обновляется
Образовательная ценность	Низкая	Средняя	Высокая
Сложность интеграции	Низкая (парсинг HTML)	Средняя (API + токены)	Средняя (API + пагинация)

Методы интеграции:

1. Автоматизированный парсинг — для TIOBE Index
2. REST API интеграция — для GitHub и Stack Overflow
3. Кэширование и периодическое обновление данных
4. Нормализация названий языков к единому формату

Выводы. Комплексное использование TIOBE Index, GitHub API и Stack Overflow API позволяет создать многомерную картину популярности и использования языков программирования. Каждый источник дополняет другие: TIOBE отражает поисковый интерес, GitHub — практическое применение, Stack Overflow — проблемы сообщества. Интеграция требует разработки специализированных модулей с учётом специфики каждого источника.

1.4. Веб-технологии для разработки электронных библиотек

Разработка современной электронной библиотеки требует комплексного подхода к выбору технологического стека, учитывающего функциональные требования системы и особенности работы с семантическими данными.

1.4.1. Backend-технологии и фреймворки

Python выбран как основной язык backend-части системы благодаря:

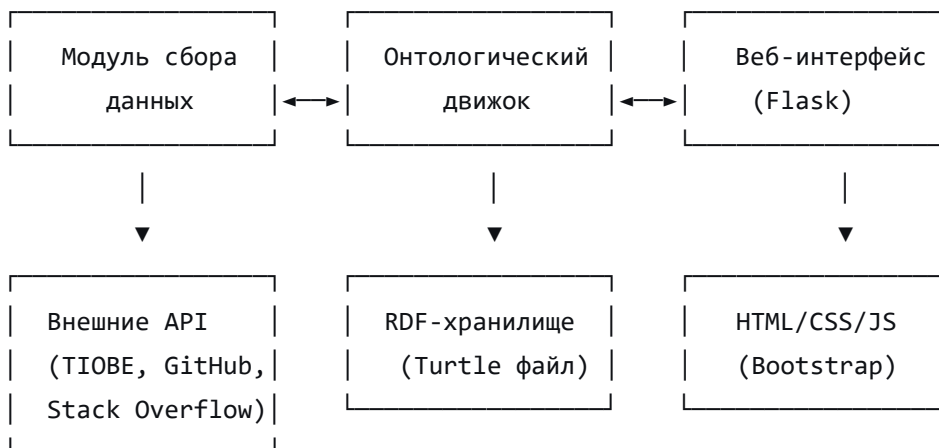
- Богатой экосистеме библиотек для работы с данными
- Поддержке семантических технологий через RDFLib
- Простоте интеграции с API различных источников
- Активному сообществу разработчиков

Ключевые Python-библиотеки:

- **RDFLib** — работа с RDF-графами
- **Flask** — микровеб-фреймворк для REST API
- **Requests** — HTTP-запросы к внешним API
- **BeautifulSoup4** — парсинг HTML-страниц
- **Matplotlib** — генерация графиков и визуализаций

Архитектурный подход: микросервисная архитектура с разделением на модули сбора данных, онтологический движок и веб-интерфейс.

text



1.4.2. Технологии работы с семантическими данными

RDF (Resource Description Framework) — основа семантического представления данных. Использован синтаксис Turtle для хранения онтологии благодаря читаемости и компактности.

SPARQL — язык запросов для извлечения данных из RDF-графов, позволяющий выполнять сложные семантические запросы.

Хранение данных: Для прототипа использовано файловое хранение в формате Turtle. Для production-систем рекомендуется переход на специализированные трипл-столе (GraphDB, Virtuoso).

1.4.3. Frontend-технологии и интерфейсы

Bootstrap 5 — CSS-фреймворк для создания адаптивного интерфейса:

- Grid-система для гибкой верстки
- Готовые компоненты для быстрой разработки
- Поддержка мобильных устройств
- Темы и кастомизация через CSS-переменные

JavaScript (ES6+) — клиентская логика:

- Fetch API для асинхронной загрузки данных
- Динамическое обновление контента
- Обработка событий и валидация форм

Font Awesome — библиотека иконок для улучшения UX.

1.4.4. Интеграция с внешними системами

REST API архитектура с JSON-форматом обмена данными:

text

GET	/api/languages	# Список всех языков
POST	/api/search	# Семантический поиск
GET	/api/chart/tiobe	# График TIOBE рейтингов
GET	/api/chart/github	# График GitHub статистики

Аутентификация: OAuth 2.0 для GitHub API, ключи приложений для Stack Overflow.

1.4.5. Технологии визуализации данных

Matplotlib и Seaborn — генерация статических графиков:

- Столбчатые диаграммы для сравнения рейтингов
- Линейные графики для отображения трендов
- Base64-кодирование для встраивания в веб-страницы

1.4.6. Инструменты разработки

- **Git** — система контроля версий
- **Virtualenv** — изоляция Python-зависимостей
- **Flask логирование** — отслеживание запросов

Сравнение технологических решений:

Категория	Выбранная технология	Обоснование выбора
Backend язык	Python 3.10+	Лучшая поддержка семантических библиотек
Веб-фреймворк	Flask	Минимализм, гибкость, легкость расширения
RDF обработка	RDFLib	Нативная интеграция с Python
Frontend фреймворк	Bootstrap 5	Адаптивность, готовые компоненты
Визуализация	Matplotlib	Простота использования, интеграция с Python

Выводы. Выбранный технологический стек представляет сбалансированное сочетание проверенных технологий и современных подходов. Python с RDFLib обеспечивает работу с семантическими данными, Flask предоставляет основу для веб-приложения, Bootstrap 5 гарантирует адаптивный интерфейс. Комплексное использование этих технологий позволяет реализовать все требуемые функции семантической библиотеки.

2. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ПРОТОТИПА СЕМАНТИЧЕСКОЙ БИБЛИОТЕКИ

2.1. Проектирование системы: требования, архитектура и онтология

Проектирование семантической библиотеки началось с анализа требований и определения ключевых сущностей предметной области. Система должна обеспечивать не только хранение и поиск информации о языках программирования, но и интеграцию динамических данных из внешних источников.

2.1.1. Функциональные требования

На основе исследования пользовательских сценариев выделены следующие функциональные требования:

1. **Управление онтологией** — создание и модификация классов, свойств, экземпляров языков
2. **Сбор данных из внешних источников** — автоматическое обновление TIOBE, GitHub, Stack Overflow данных
3. **Семантический поиск** — полнотекстовый поиск с фильтрацией и ранжированием

4. **Визуализация данных** — графическое представление рейтингов и статистики
5. **Управление публикациями** — добавление и категоризация книг, статей, документации

2.1.2. Архитектура системы

Архитектура спроектирована по модульному принципу с четким разделением ответственности между компонентами.

text

[ВСТАВИТЬ РИСУНОК: Архитектура системы (рис. 2.1)]

Рис. 2.1. Архитектура семантической библиотеки

Ключевые компоненты:

1. **Веб-интерфейс (Flask)** — обработка HTTP-запросов, REST API, HTML-страницы
2. **Бизнес-логика** — поисковый движок, менеджер онтологии, менеджер публикаций
3. **Модули интеграции** — клиенты для TIOBE, GitHub, Stack Overflow API
4. **Хранилище данных** — RDF-файл в формате Turtle

2.1.3. Онтология предметной области

Онтология разработана с использованием методологии Methontology и инструмента Protégé.

Иерархия основных классов:

text

Thing

├─ Agent

| └─ Author

├─ Document

| └─ Publication

| | └─ Book

| | └─ Article

| | └─ Documentation

| | └─ Tutorial

| └─ Framework

└─ ProgrammingLanguage

└─ Python

└─ Java

└─ JavaScript

└─ C

└─ CPlusPlus

└─ CSharp

Основные свойства:

- **Object properties:** aboutLanguage, writtenBy, hasParadigm, hasFramework
- **Data properties:** tiobeRank, tiobeRating, githubRepositories, githubStars, stackOverflowQuestions

2.1.4. Диаграмма вариантов использования

text

[ВСТАВИТЬ РИСУНОК: Диаграмма вариантов использования (рис. 2.2)]

Рис. 2.2. Основные сценарии взаимодействия пользователя с системой

Основные сценарии использования:

- Поиск информации о языках программирования
- Просмотр и фильтрация данных
- Визуализация статистики
- Обновление данных из внешних источников

2.1.5. Обеспечение целостности данных

Реализованы механизмы:

1. **Нормализация данных** — приведение названий языков к единому формату
2. **Валидация входных данных** — проверка форматов, URL, уникальности
3. **Обработка ошибок API** — повторные запросы, демо-данные, логирование

Выводы. Проектирование системы началось с глубокого анализа требований и разработки онтологии, ставшей основой для всей системы. Модульная архитектура обеспечила гибкость и масштабируемость решения. Использование стандартов семантического веба (RDF, OWL, SPARQL) гарантировало интероперабельность системы с другими семантическими приложениями.

2.2. Проектирование модели данных и структуры каталога

Эффективная организация данных является ключевым аспектом библиотечной системы. Для семантической библиотеки по программированию разработана комплексная модель данных, объединяющая статические характеристики языков и динамические метрики из внешних источников.

2.2.1. Модель данных для языков программирования

Основная сущность системы — язык программирования — описана следующими атрибутами:

Идентификационные данные:

- Название языка (Python, Java, JavaScript)
- Отображаемое название (C++, C#)

- Краткое описание

Технические характеристики:

- Год первого появления
- Парадигмы программирования
- Система типизации
- Поддерживаемые платформы

Рейтинги и метрики:

- Позиция в рейтинге TIOBE
- Количество репозиторий на GitHub
- Количество вопросов на Stack Overflow

Метаданные:

- Дата последнего обновления
- Источник данных

2.2.2. Структура каталога публикаций

Каталог организован по иерархическому принципу с поддержкой множественной категоризации:

text

[ВСТАВИТЬ РИСУНОК: Структура каталога публикаций (рис. 2.3)]

Рис. 2.3. Иерархическая организация каталога публикаций

Основные категории:

1. **По типу ресурса:** книги, статьи, документация, tutorиалы
2. **По языку программирования:** Python, Java, JavaScript, C, C++, C#
3. **По уровню сложности:** начальный, средний, продвинутый
4. **По языку изложения:** русский, английский

Модель данных публикации включает:

- Обязательные поля: название, тип, язык программирования
- Опциональные поля: автор, ISBN, URL, год издания, ключевые слова
- Метаданные: дата добавления, источник

2.2.3. RDF-представление модели данных

Модель реализована в формате RDF/Turtle. Каждая сущность описывается как набор триплетов, связывающих субъект, предикат и объект.

Пример структуры данных:

- Язык программирования описывается классом `ProgrammingLanguage`
- Публикации связываются с языками через свойство `aboutLanguage`
- Метрики из внешних источников хранятся как свойства данных

2.2.4. Связи между сущностями

Для обеспечения семантических связей определены объектные свойства:

text

[ВСТАВИТЬ РИСУНОК: Схема связей между сущностями (рис. 2.4)]

Рис. 2.4. Семантические связи между основными сущностями онтологии

Основные связи:

- `aboutLanguage` — связь публикации с языком программирования
- `writtenBy` — указание автора публикации
- `hasFramework` — связь языка с фреймворками
- `usedInDomain` — области применения языка

2.2.5. Нормализация и индексация данных

Нормализация данных:

- Приведение текста к нижнему регистру
- Удаление специальных символов
- Нормализация названий языков (C++ → cpp, C# → csharp)

Индексация для поиска:

- Индекс по названиям публикаций
- Индекс по ключевым словам
- Индекс по авторам
- Индекс по языкам программирования

2.2.6. Структура хранения данных

Данные организованы в виде связанных файлов:

text

data/

```
├─ programming_languages.ttl    # Основная онтология
├─ publications/                # Каталог публикаций
│   ├─ books.ttl
│   ├─ articles.ttl
│   ├─ documentation.ttl
│   └─ tutorials.ttl
├─ indices/                    # Поисковые индексы
└─ metadata/                   # Метаданные системы
```

2.2.7. Механизмы целостности и валидации

Валидация данных:

- Проверка обязательных полей
- Валидация форматов данных (числа, даты, URL)
- Проверка уникальности идентификаторов

Обработка ошибок:

- Валидация входных данных
- Обработка ошибок API
- Логирование для последующего анализа

2.2.8. Оптимизация производительности

Кэширование:

- Кэширование часто используемых данных
- LRU-кэш для запросов к онтологии
- Кэширование результатов поиска

Пакетная обработка:

- Обработка публикаций пакетами
- Периодическое сохранение для избежания потерь данных
- Оптимизация использования памяти

Выводы. Разработанная модель данных и структура каталога обеспечивают гибкое и эффективное хранение информации о языках программирования.

Использование семантических технологий позволило создать богатую сеть связей между сущностями, что открывает возможности для сложных запросов и анализа данных. Реализованные механизмы валидации и оптимизации гарантируют целостность данных и производительность системы.

2.3. Реализация базовых модулей управления данными библиотеки

Реализация системы управления данными основана на модульном подходе, где каждый компонент отвечает за определенный аспект работы с семантическими данными.

2.3.1. Модуль управления онтологией (OntologyManager)

Основной компонент системы, обеспечивающий взаимодействие с RDF-графом.

Ключевые функции:

- Загрузка и сохранение онтологии из файла в формате Turtle

- Добавление новых языков программирования в онтологию
- Обновление рейтингов и метрик языков (TIOBE, GitHub, Stack Overflow)
- Управление пространствами имен и метаданными

Основные методы:

1. `add_programming_language()` — добавление языка с указанием названия и описания
2. `update_language_ranking()` — обновление TIOBE рейтинга для существующего языка
3. `save_ontology()` — сохранение текущего состояния онтологии в файл

2.3.2. Модуль управления публикациями (PublicationManager)

Отвечает за работу с публикациями и их связью с языками программирования.

Функциональность:

- Добавление различных типов публикаций (книги, статьи, документация, tutoriales)
- Связывание публикаций с языками программирования через свойство `aboutLanguage`
- Управление метаданными публикаций (автор, ISBN, URL, год издания, ключевые слова)
- Генерация уникальных идентификаторов для публикаций

Вспомогательные методы:

- `_generate_publication_id()` — генерация уникальных ID на основе названия и типа
- `get_publications_for_language()` — получение всех публикаций для указанного языка

2.3.3. Модуль валидации данных (DataValidator)

Обеспечивает проверку целостности и корректности данных перед добавлением в систему.

Типы валидации:

1. **Валидация данных языка программирования:**
 - Проверка обязательных полей (название, отображаемое название)
 - Валидация рейтингов TIOBE (целое число от 1 до 100)
 - Проверка формата процентного рейтинга
 - Валидация URL-адресов
2. **Валидация данных публикации:**
 - Проверка обязательных полей для каждого типа публикации
 - Валидация URL для статей и документации
 - Проверка наличия автора для книг

2.3.4. Модуль работы с авторами (AuthorManager)

Управляет информацией об авторах публикаций и их связями с публикациями.

Функции:

- Создание новых авторов или получение существующих
- Нормализация имен авторов для использования в URI
- Получение всех публикаций конкретного автора
- Управление метаданными авторов

2.3.5. Модуль статистики (StatisticsManager)

Предоставляет статистическую информацию о данных в системе.

Собираемая статистика:

- Общее количество языков программирования
- Количество языков с данными из каждого источника (TIOBE, GitHub, Stack Overflow)
- Распределение публикаций по типам (книги, статьи, документация, tutorиалы)
- Время последнего обновления данных

2.3.6. Взаимодействие модулей

Модули организованы в единую систему с четким разделением ответственности:

text

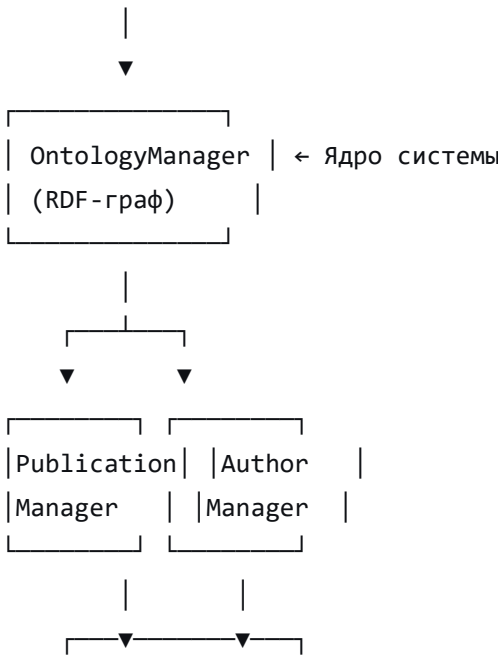
[ВСТАВИТЬ РИСУНОК: Взаимодействие модулей системы (рис. 2.5)]

Рис. 2.5. Архитектура взаимодействия модулей управления данными

Архитектурный паттерн:

text

Основное приложение



| DataValidator |

Пример рабочего процесса:

1. Инициализация менеджеров онтологии, публикаций и авторов
2. Валидация входных данных с помощью DataValidator
3. Добавление языка программирования через OntologyManager
4. Добавление связанных публикаций через PublicationManager
5. Сбор статистики через StatisticsManager
6. Сохранение изменений в онтологии

Выводы. Реализованные базовые модули управления данными обеспечивают полный цикл работы с семантической библиотекой: от добавления и валидации данных до их извлечения и статистического анализа. Модульная архитектура позволяет легко расширять функциональность системы. Использование единого интерфейса OntologyManager для работы с RDF-графом гарантирует согласованность данных и упрощает интеграцию различных компонентов системы.

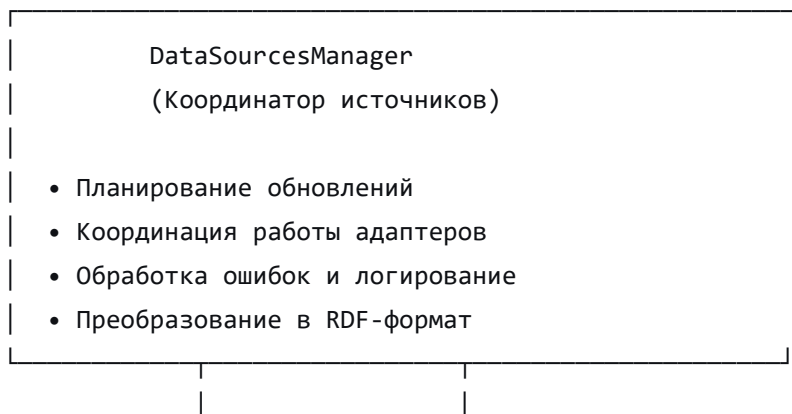
2.4. Реализация модулей интеграции с внешними источниками данных

Интеграция с внешними источниками данных является ключевым компонентом системы, обеспечивающим актуальность информации о языках программирования. Реализованы три основных модуля интеграции: парсер TIOBE Index, клиент GitHub API и клиент Stack Overflow API.

2.4.1. Архитектура модулей интеграции

Система интеграции построена по паттерну "**Адаптер**", обеспечивая единый интерфейс работы с разнородными внешними источниками. Каждый источник данных имеет собственный адаптер, который преобразует внешний формат данных во внутренний RDF-формат семантической библиотеки.

text



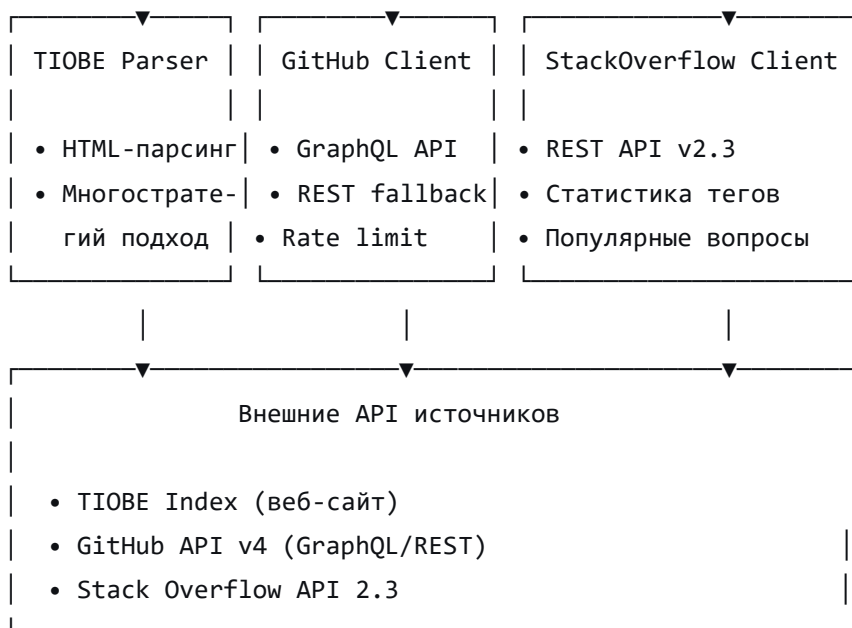


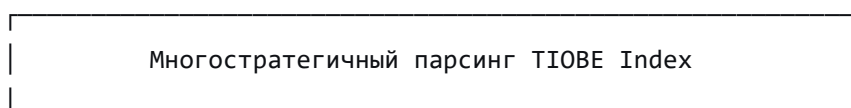
Рисунок 2.4.1: Архитектура системы интеграции с внешними источниками данных

2.4.2. Модуль парсинга TIOBE Index

Модуль **AdvancedTIOBEParser** реализует продвинутый парсинг данных с официального сайта TIOBE Index, использующий многостратегичный подход для обеспечения отказоустойчивости при изменениях структуры сайта.

Ключевые особенности:

1. **Многостратегичный парсинг** - последовательное применение различных методов извлечения данных:
 - Прямой парсинг HTML через CSS-селекторы
 - Поиск по текстовым паттернам
 - Извлечение данных из JavaScript-скриптов
 - Использование альтернативных источников и архивов
 - Резервные демо-данные при полном отказе
2. **Адаптивные алгоритмы** обнаружения таблиц рейтинга, учитывающие возможные изменения структуры страницы TIOBE.
3. **Интеллектуальный анализ ячеек таблицы**, автоматически определяющий назначение колонок (ранг, название языка, рейтинг).
4. **Система нормализации названий**, преобразующая внешние обозначения в стандартизированные имена (например, "C#" → "CSharp", "C++" → "CPlusPlus").
text



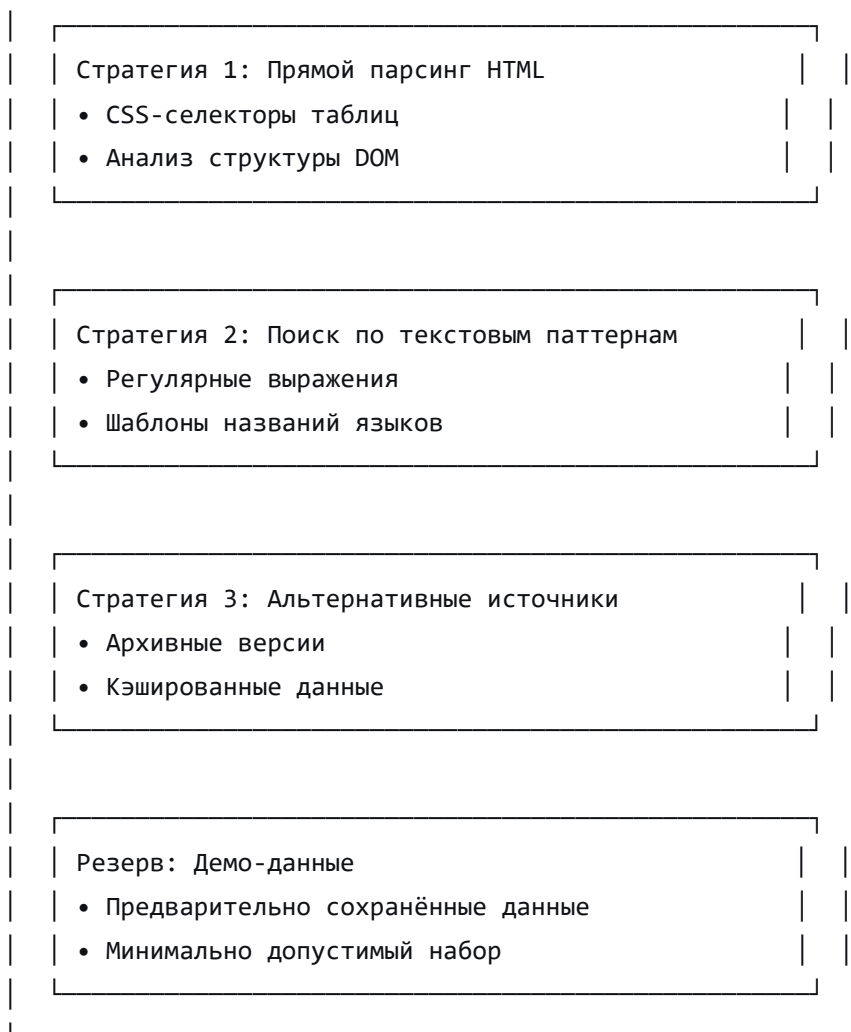


Рисунок 2.4.2: Схема многостратегического парсинга TIOBE Index

2.4.3. Модуль интеграции с GitHub API

Модуль **GitHubAPIClient** обеспечивает взаимодействие с GitHub API для получения актуальной статистики по языкам программирования, используя преимущества как GraphQL, так и REST API.

Основные возможности:

1. **Приоритетное использование GraphQL API**, позволяющее получать необходимые данные в одном запросе и сокращающее количество обращений к API.
2. **Интеллектуальный fallback на REST API** при недоступности или ошибках GraphQL.
3. **Комплексная статистика** для каждого языка:
 - Общее количество репозиторий
 - Популярные репозитории (топ-5 по звёздам)
 - Среднее количество звёзд на репозиторий

- Активность обновления
4. Система контроля **rate limits** с автоматическими паузами между запросами и мониторингом оставшихся запросов.
- text

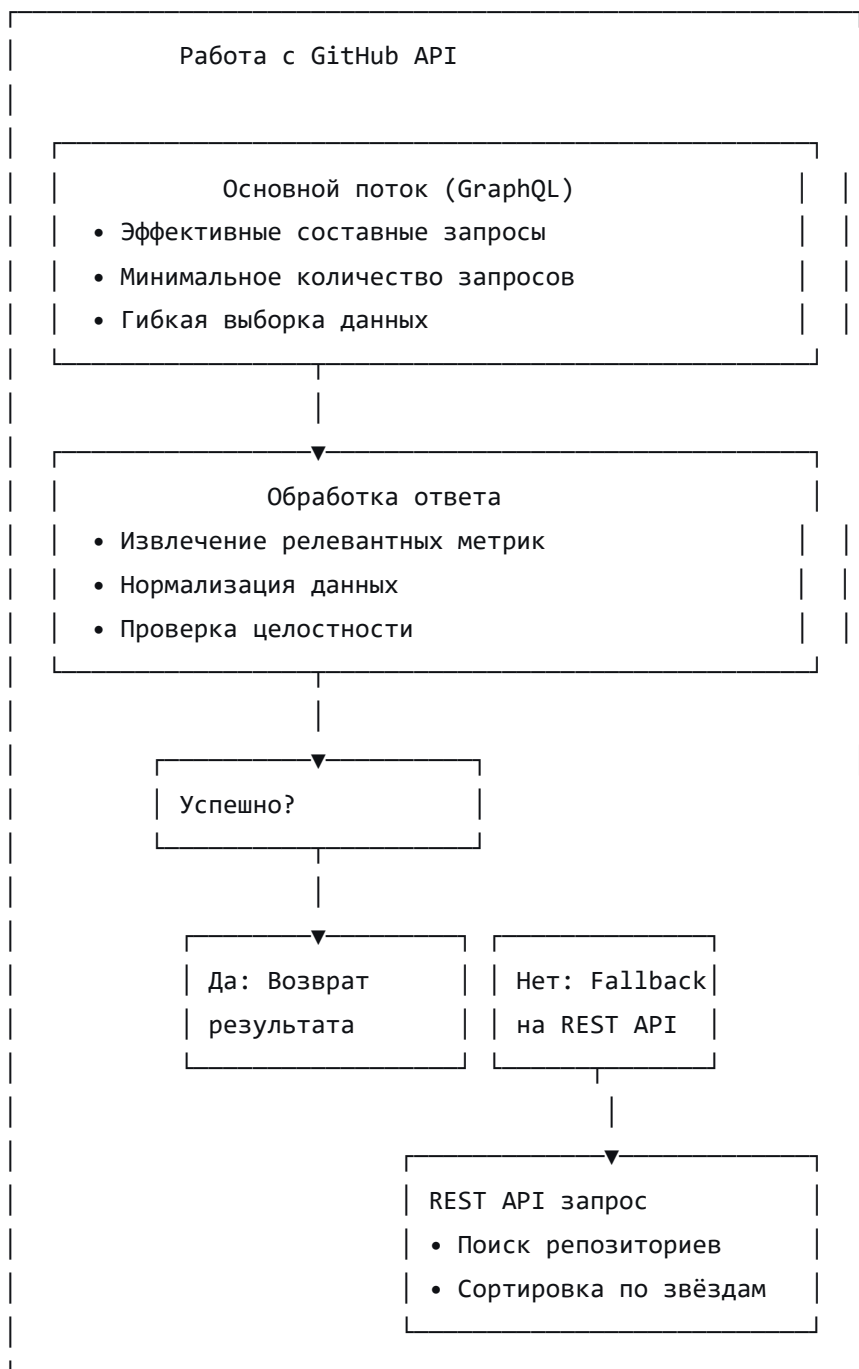


Рисунок 2.4.3: Схема работы с GitHub API

2.4.4. Модуль интеграции с Stack Overflow API

Модуль **StackOverflowAPIClient** взаимодействует со Stack Overflow API 2.3 для получения статистики по тегам языков программирования и анализа популярных вопросов.

Функциональность модуля:

1. **Статистика по тегам языков:**
 - Общее количество вопросов с тегом
 - Активность тега (дата последнего вопроса)
 - Связанные теги и технологии
 - Статус тега (обязательный, модераторский)
2. **Анализ популярных вопросов** за определённый период с возможностью настройки временного интервала и количества вопросов.
3. **Интеграция с системой квот API**, обеспечивающая соблюдение ограничений Stack Overflow.
text

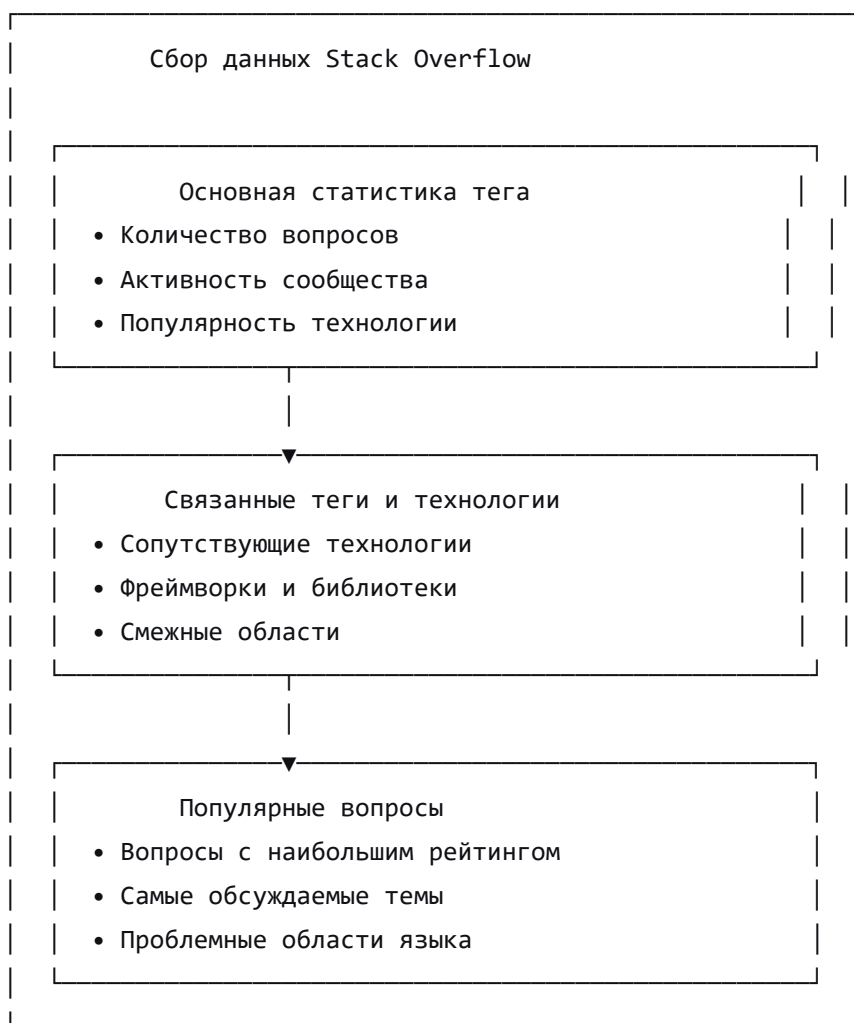


Рисунок 2.4.4: Структура данных Stack Overflow

2.4.5. Менеджер источников данных (DataSourcesManager)

DataSourcesManager является центральным координатором системы интеграции, управляющим всем процессом сбора и обновления данных из внешних источников.

Основные функции:

1. Координация обновлений:

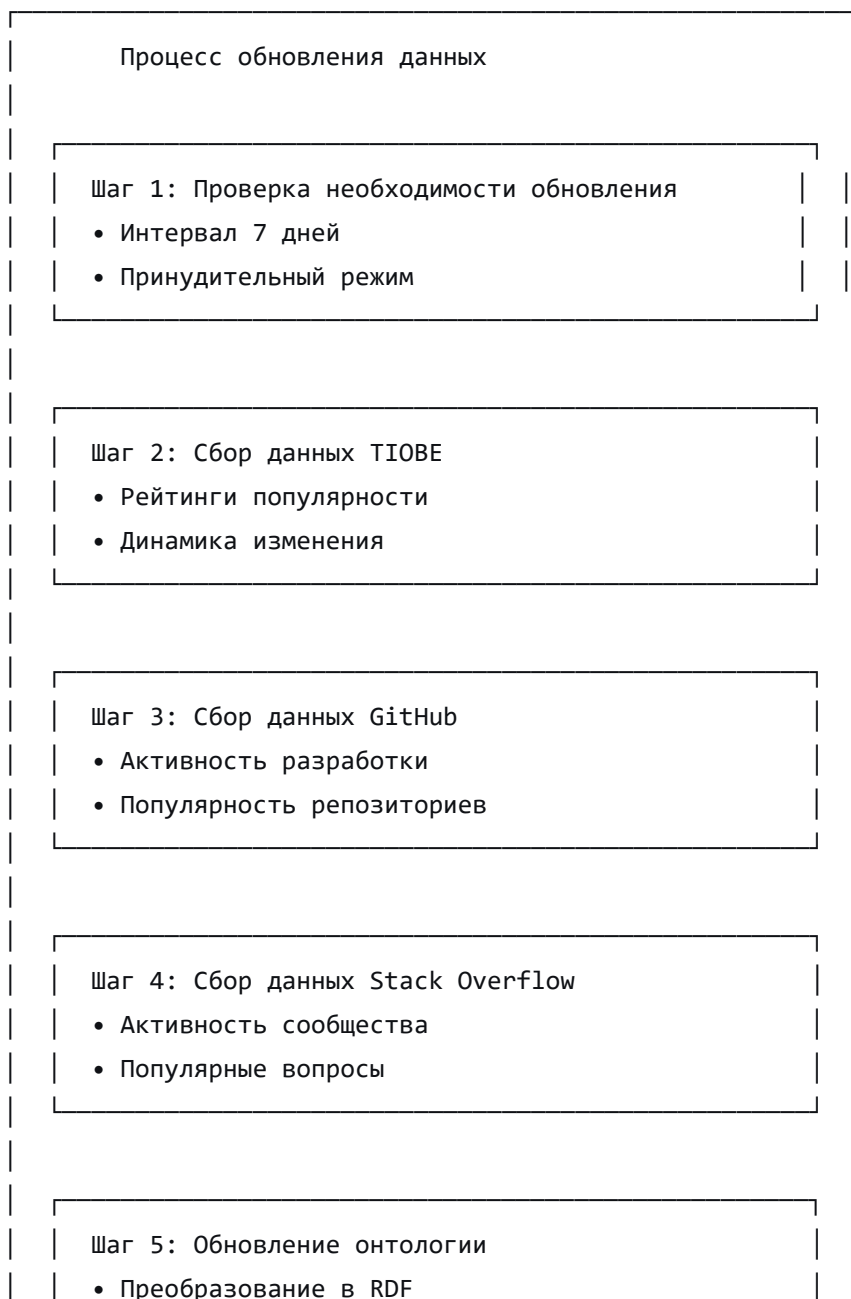
- Планирование периодических обновлений (по умолчанию раз в 7 дней)
- Последовательный сбор данных из трёх источников
- Принудительное обновление по требованию
- Отслеживание времени последнего успешного обновления

2. Интеграция с онтологией:

- Преобразование данных внешних источников в RDF-триплеты
- Обновление существующих записей в семантической библиотеке
- Обеспечение целостности данных при обновлении

3. Обработка целевых языков, позволяющая гибко настраивать список отслеживаемых языков программирования.

text



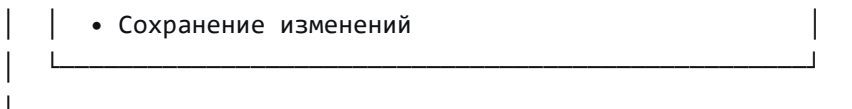


Рисунок 2.4.5: Последовательность обновления данных

2.4.6. Обработка ошибок и мониторинг

Система включает комплексный механизм обработки ошибок и мониторинга, обеспечивающий надёжность работы интеграционных модулей.

Компоненты системы:

1. Логирование операций:

- Запись всех операций обновления в структурированном формате (JSON Lines)
- Отслеживание статуса выполнения каждой операции
- Детализация ошибок и исключений

2. Система уведомлений с градацией по уровням важности:

- Информационные сообщения о успешных операциях
- Предупреждения о потенциальных проблемах
- Ошибки критических сбоев

3. Механизм повторов для временных сбоев подключения к внешним API.

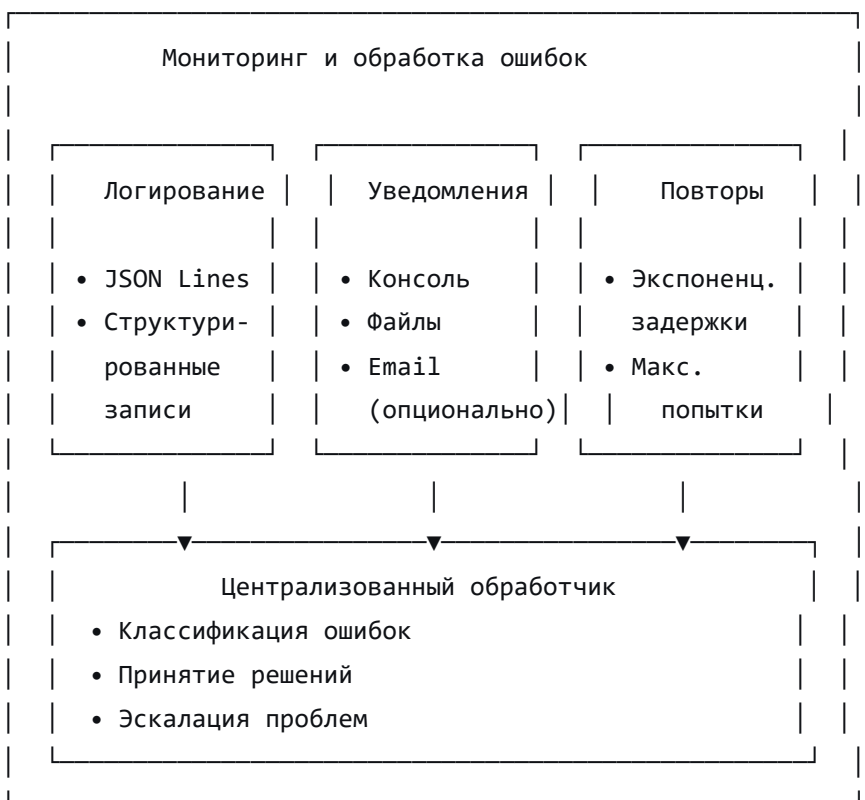


Рисунок 2.4.6: Архитектура системы мониторинга

2.4.7. Оптимизации и производительность

Для обеспечения высокой производительности и снижения нагрузки на внешние API реализованы следующие оптимизации:

- 1. **Кэширование запросов** на диске с настраиваемым временем жизни кэша.
- 2. **Интеллектуальная стратегия запросов**, минимизирующая количество обращений к API.
- 3. **Пакетная обработка** данных для языков программирования.
- 4. **Фоновое выполнение** длительных операций обновления.

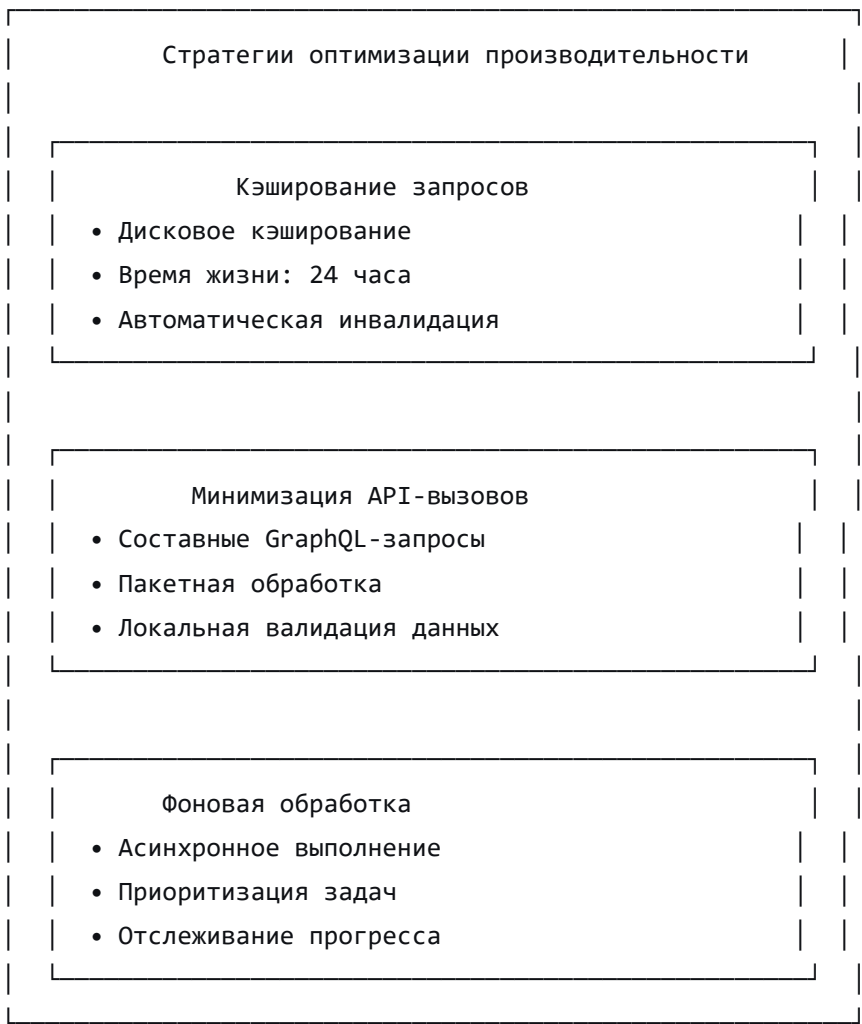


Рисунок 2.4.7: Оптимизации производительности

2.4.8. Выводы

Реализованная система интеграции с внешними источниками данных представляет собой **надёжную, отказоустойчивую и высокопроизводительную платформу** для сбора актуальной информации о языках программирования. Ключевые достижения:

1. **Гибкость и адаптивность** благодаря многостратегичному подходу к парсингу и работе с API.
2. **Отказоустойчивость** через реализацию механизмов fallback, повторов и резервных источников данных.
3. **Масштабируемость** модульной архитектуры, позволяющей легко добавлять новые источники данных.
4. **Эффективность** использования ресурсов за счёт кэширования, оптимизации запросов и фоновой обработки.
5. **Комплексный мониторинг** и логирование всех операций системы.

Система успешно решает задачу поддержания актуальности семантической библиотеки, обеспечивая регулярное и надёжное обновление данных из авторитетных внешних источников.

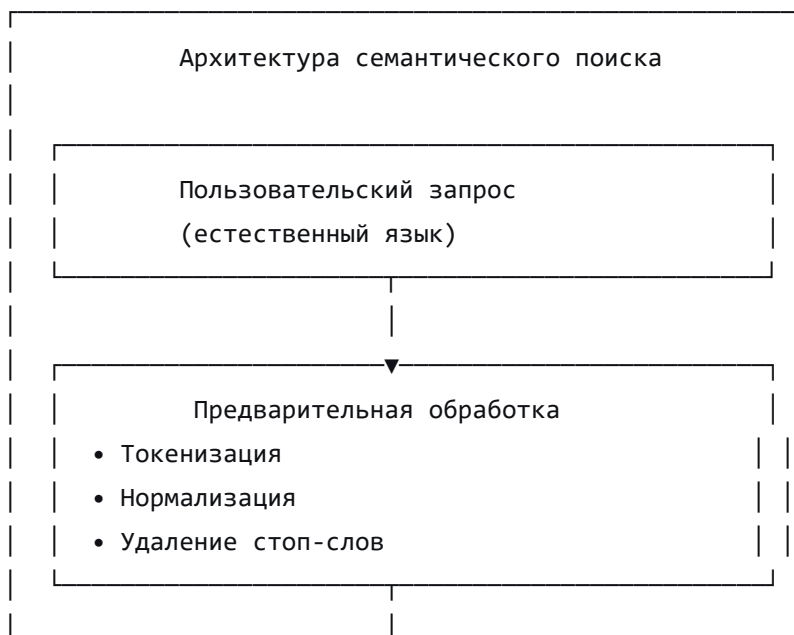
2.5. Реализация модуля семантического поиска

Модуль семантического поиска является ключевым компонентом системы, обеспечивающим интеллектуальный доступ к данным библиотеки. В отличие от традиционного полнотекстового поиска, семантический поиск учитывает смысл запросов и связи между сущностями в онтологии.

2.5.1. Архитектура модуля поиска

Модуль семантического поиска построен по **многоуровневой архитектуре**, преобразующей пользовательские запросы на естественном языке в структурированные SPARQL-запросы к RDF-онтологии. В отличие от традиционного полнотекстового поиска, система учитывает семантические связи между сущностями.

text



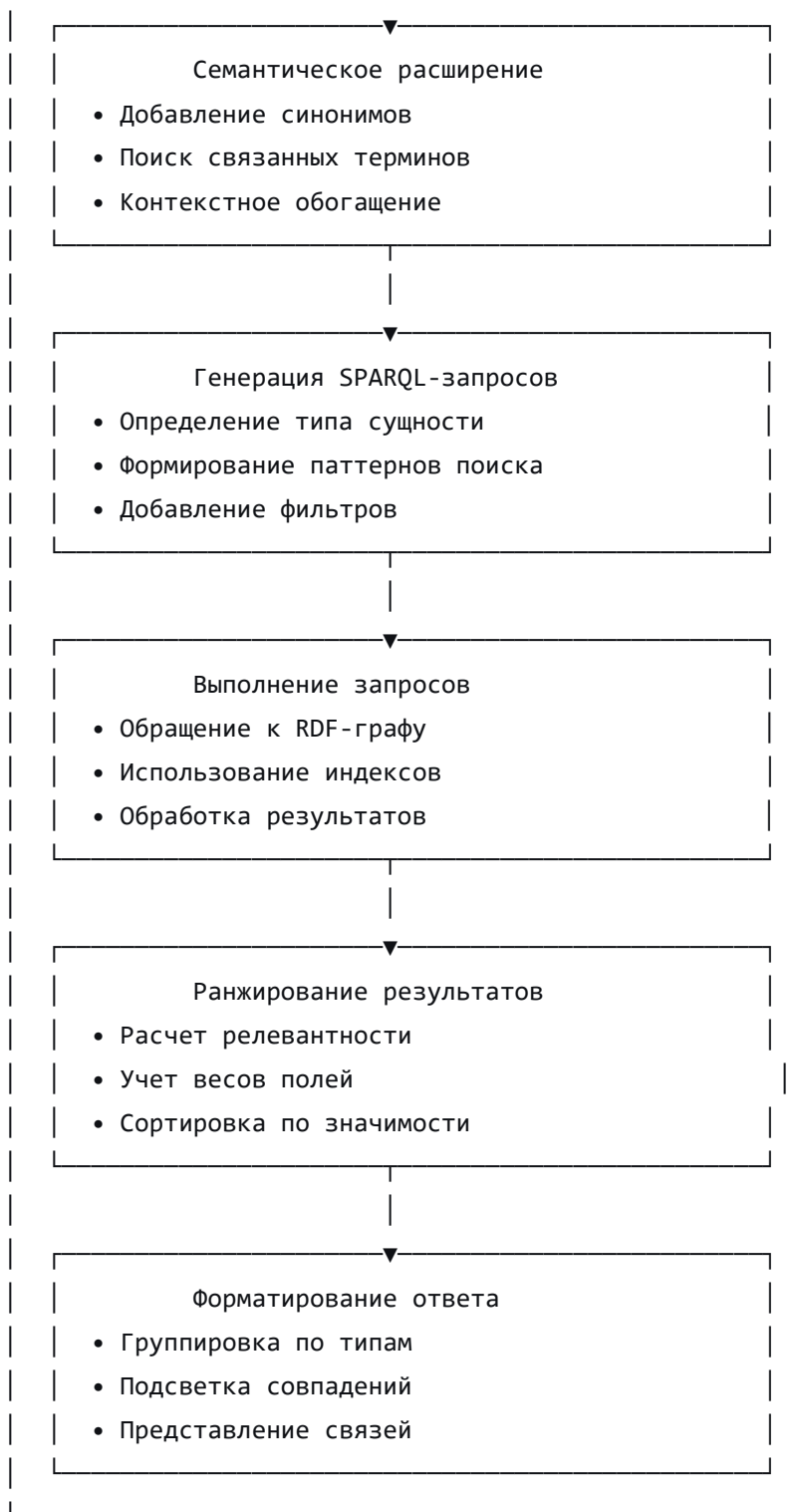


Рисунок 2.5.1: Общая архитектура модуля семантического поиска

2.5.2. Класс `OntologySearch`

Основной класс `OntologySearch` интегрирует все компоненты поисковой системы:

- **Загрузка онтологии** и привязка пространств имён
- **Инициализация компонентов** обработки запросов

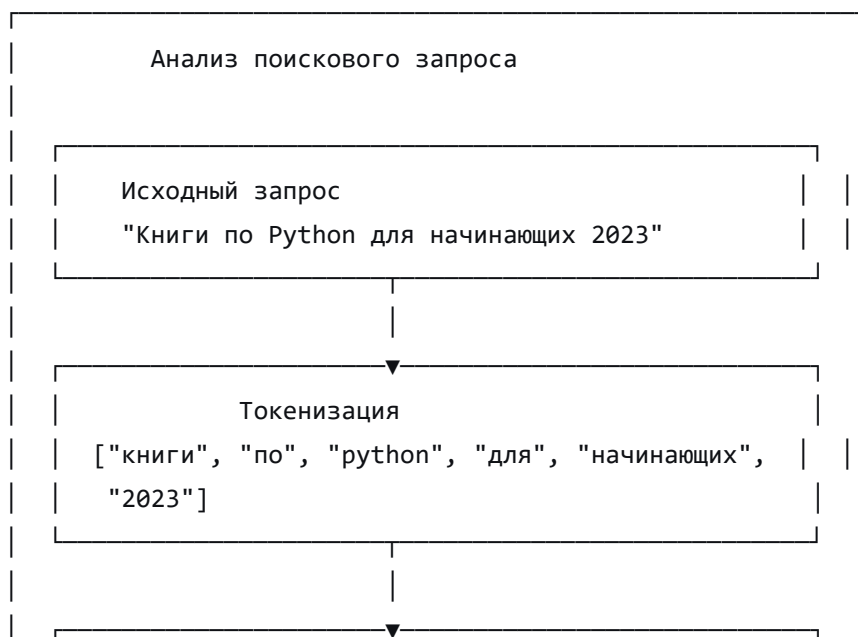
- **Управление кэшированием** для повышения производительности
- **Координация** всего процесса поиска

2.5.3. Анализатор запросов (QueryAnalyzer)

Анализатор выполняет глубокую обработку естественно-языковых запросов:

Функциональность:

1. **Токенизация** с учётом специфики терминологии программирования:
 - Обработка составных терминов (C++, C#)
 - Разделение по пробелам, запятым, дефисам и подчёркиваниям
2. **Нормализация** токенов:
 - Приведение к стандартным названиям языков (python3 → python, js → javascript)
 - Удаление специальных символов с сохранением семантики
3. **Определение типа запроса** через анализ ключевых слов:
 - Запросы о языках программирования
 - Поиск книг и учебных материалов
 - Поиск статей и научных публикаций
 - Поиск tutorиалов и руководств
 - Поиск фреймворков и библиотек
 - Поиск документации
4. **Извлечение фильтров** из запроса:
 - Языки программирования
 - Уровень сложности (начальный, средний, продвинутый)
 - Год публикации
 - Автор



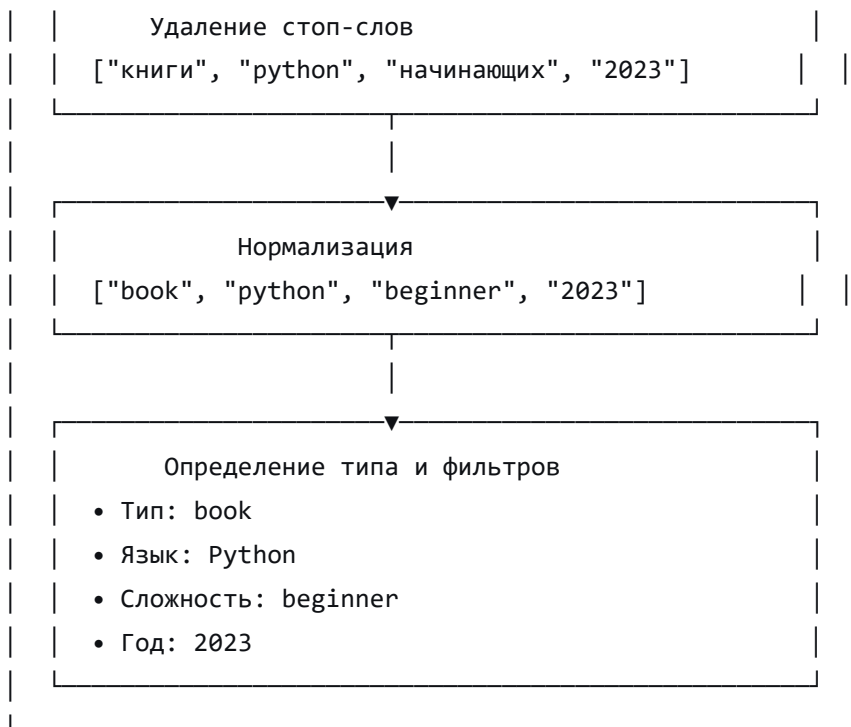


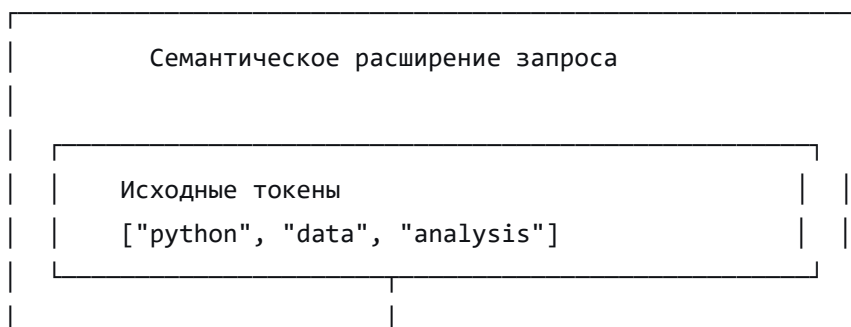
Рисунок 2.5.2: Процесс анализа поискового запроса

2.5.4. Расширитель запросов (QueryExpander)

Модуль расширения обогащает запросы для повышения полноты поиска:

Механизмы расширения:

1. **Добавление синонимов** для терминов программирования:
 - Python → питон, пайтон
 - JavaScript → JS, ECMAScript
 - C# → CSharp, C Sharp
2. **Контекстное расширение** для общих понятий:
 - web → JavaScript, HTML, CSS, React, Angular
 - data → Python, R, pandas, numpy, machine learning
 - mobile → Java, Kotlin, Swift, React Native
3. **Связывание терминов** с технологическим стеком text



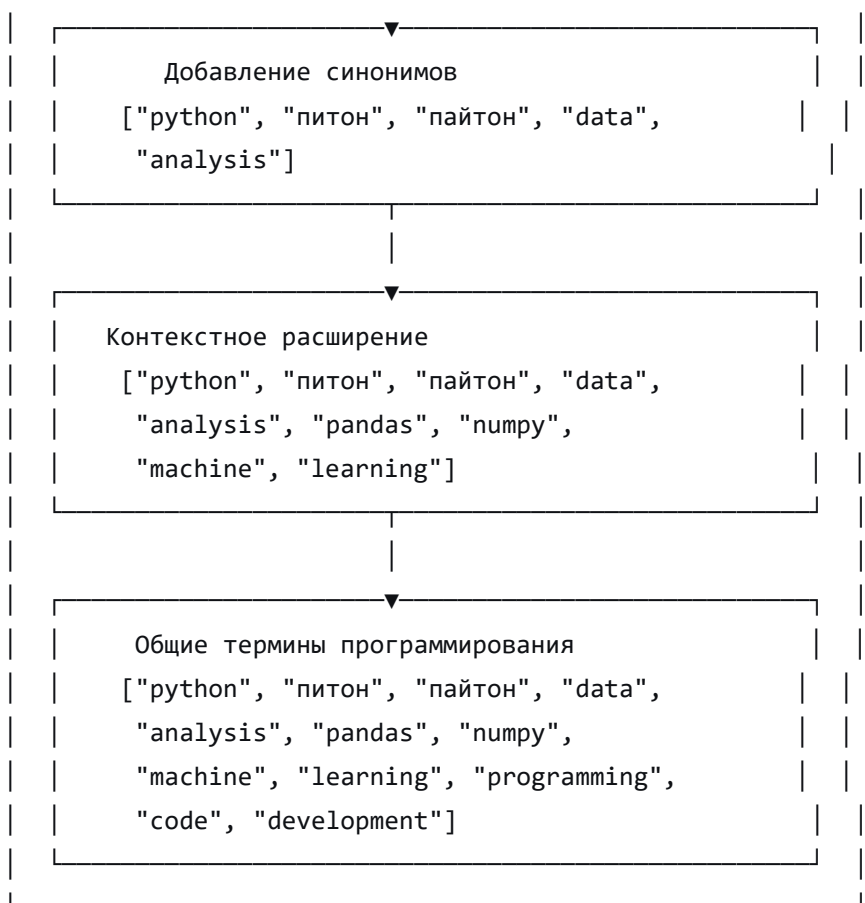


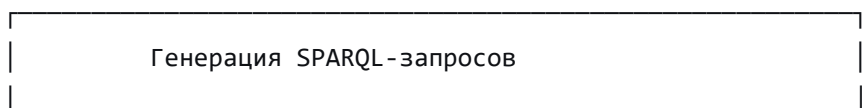
Рисунок 2.5.3: Процесс семантического расширения

2.5.5. Генератор SPARQL-запросов

Генератор преобразует обработанные запросы в формализованные SPARQL-запросы:

Особенности генерации:

1. **Динамическое формирование SELECT-части** с учётом требуемых полей
2. **Интеллектуальная WHERE-часть:**
 - Добавление паттернов в зависимости от типа сущности
 - Условные OPTIONAL блоки для необязательных свойств
 - Контекстно-зависимые условия поиска
3. **Фильтрация по типам сущностей** на основе анализа запроса
4. **Расчёт релевантности** через встроенные функции SPARQL:
 - Взвешивание совпадений в разных полях
 - Бонусы за точные совпадения



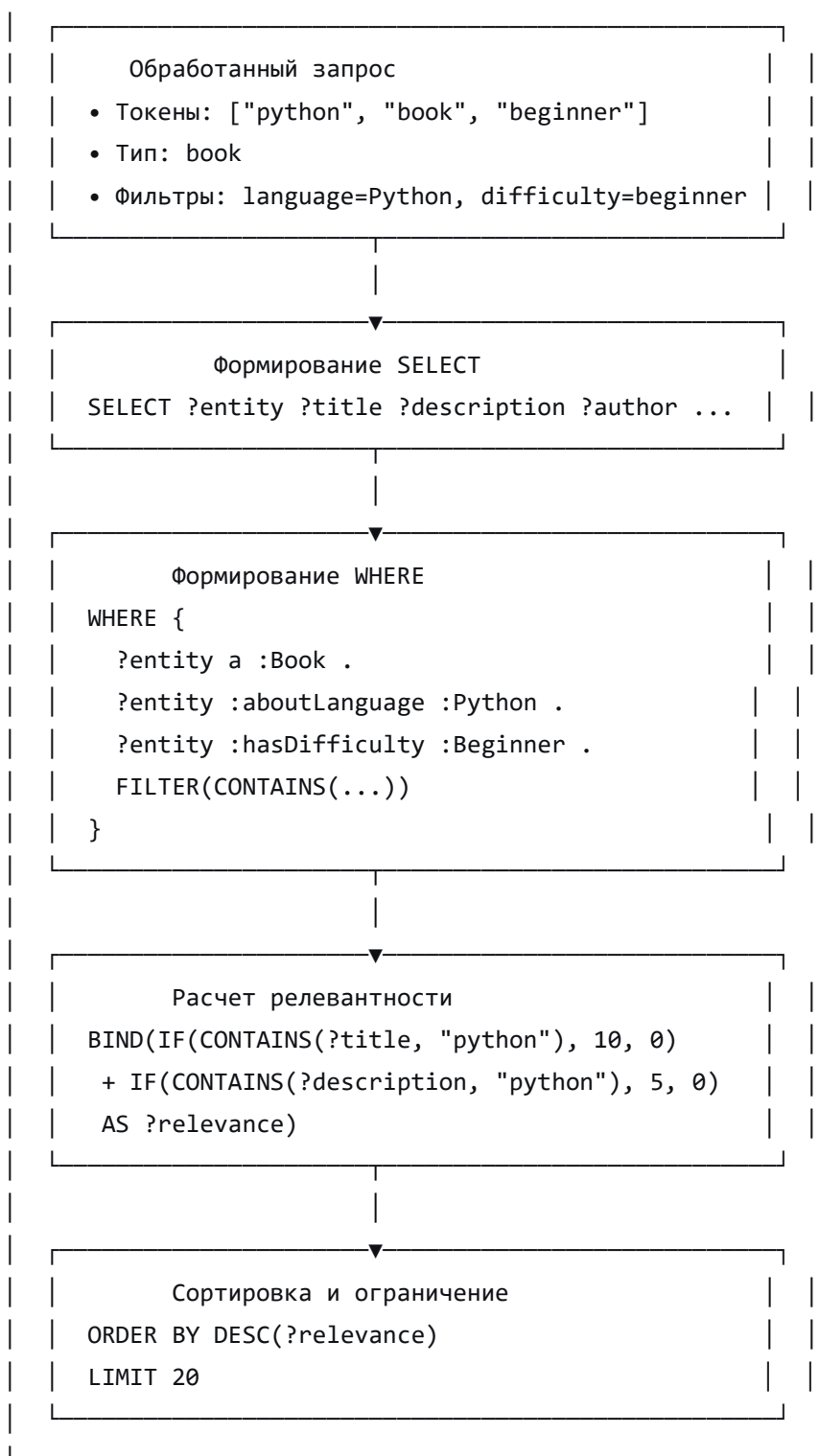


Рисунок 2.5.4: Процесс генерации SPARQL-запросов

2.5.6. Основной метод поиска

Интеграционный метод `search_by_keywords()` объединяет все этапы обработки:

text

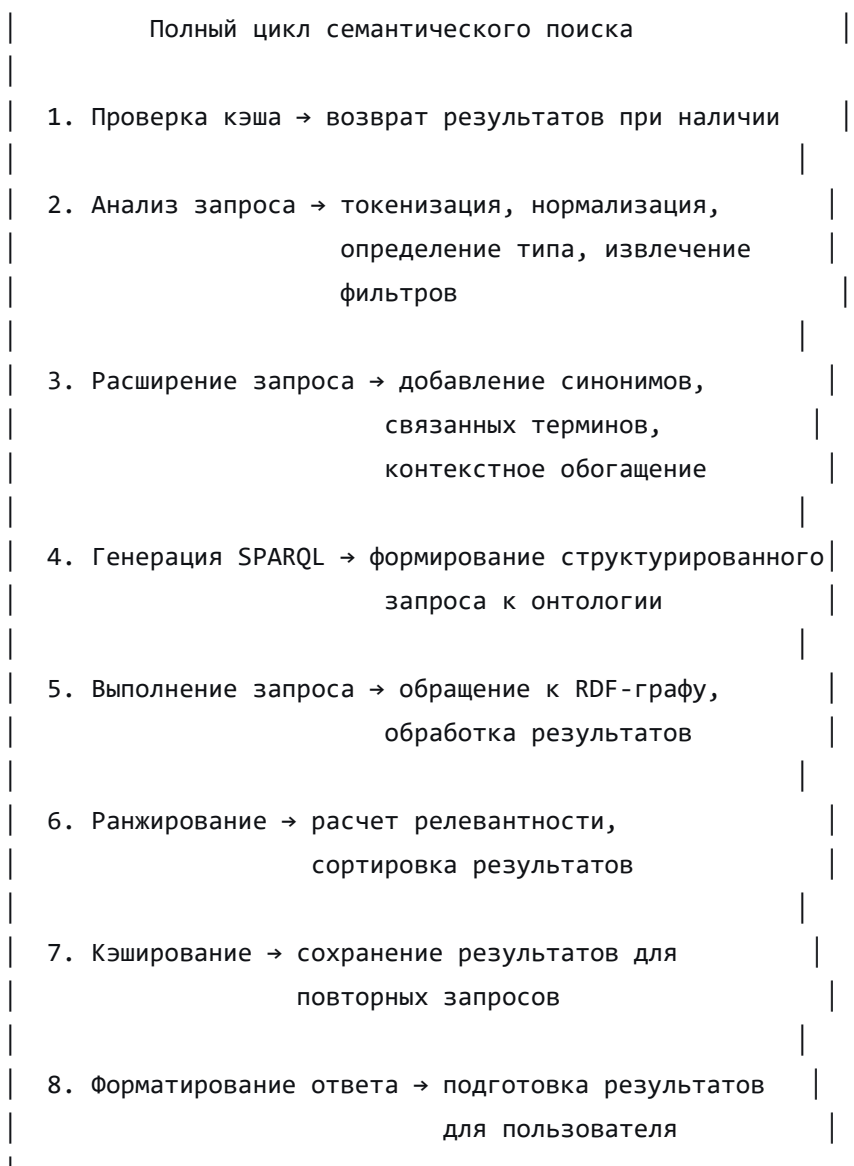


Рисунок 2.5.5: Полный цикл обработки поискового запроса

2.5.7. Ранжировщик результатов (ResultRanker)

Ранжировщик определяет релевантность найденных результатов через многофакторную систему оценки:

Факторы релевантности:

1. **Вес полей** (титул важнее описания):
 - Название: 10.0 баллов
 - Ключевые слова: 5.0 баллов
 - Описание: 2.0 балла
 - Язык программирования: 3.0 балла
 - Автор: 1.0 балл

2. Бонусы за типы сущностей:

- Язык программирования: +20.0 баллов
- Книга: +15.0 баллов
- Фреймворк: +12.0 баллов
- Документация: +10.0 баллов
- Статья/Тutorial: +8.0 баллов

3. Премия за точные совпадения (×2 веса)

4. Мультиполевой бонус за совпадение в нескольких полях

2.5.8. Специализированные методы поиска

Поиск по языку программирования

Специализированный метод `search_by_language()` предоставляет комплексную информацию:

1. **Базовая информация** о языке (парадигма, версия, создатель)
2. **Связанные ресурсы:**
 - Книги и учебные материалы
 - Фреймворки и библиотеки
 - Тutorials и документация
3. **Статистика** из внешних источников:
 - Рейтинг TIOBE
 - Активность на GitHub
 - Популярность на Stack Overflow
4. **Связанные языки** по различным критериям:
 - Синтаксическая схожесть
 - Области применения
 - Технологический стек

2.5.9. Кэширование и оптимизация

Для повышения производительности реализована система кэширования:

Особенности кэширования:

1. **Генерация ключей кэша** на основе параметров запроса (MD5-хеш)
2. **LRU-стратегия** (Least Recently Used) для управления размером кэша
3. **Автоматическая инвалидация** при изменении онтологии
4. **Статистика попаданий** для анализа эффективности кэширования

2.5.10. Метрики и мониторинг поиска

Система сбора метрик `SearchMetrics` позволяет анализировать использование поиска:

Собираемые метрики:

1. Общая статистика:

- Количество успешных/неудачных запросов
- Среднее время ответа
- Распределение по типам запросов

2. Аналитика запросов:

- Популярные поисковые фразы
- Запросы с нулевыми результатами
- Эффективность расширения запросов

3. Производительность:

- Время выполнения каждого этапа
- Эффективность кэширования
- Загрузка системы

2.5.11. Примеры использования

Пример 1: Комплексный поиск учебных материалов

text

Запрос: "Продвинутые книги по машинному обучению на Python"

Этапы обработки:

1. Анализ → ["продвинутые", "книги", "машинное", "обучение", "python"]
2. Нормализация → ["advanced", "book", "machine", "learning", "python"]
3. Определение типа → book
4. Извлечение фильтров → язык: Python, сложность: advanced
5. Расширение → + "ML", "AI", "pandas", "scikit-learn"
6. Генерация SPARQL → запрос с фильтрами по типу, языку и сложности
7. Ранжирование → книги с "advanced" и "machine learning" в названии

Пример 2: Поиск информации о языке

text

Запрос: "Сравнение TypeScript и JavaScript"

Этапы обработки:

1. Определение как специализированного запроса о языках
2. Поиск информации о TypeScript и JavaScript отдельно
3. Извлечение сравнительных характеристик:
 - Система типов

- Совместимость с ECMAScript
 - Популярность и использование
 - Рекомендуемые области применения
4. Формирование сравнительной таблицы

2.5.12. Выводы

Реализованный модуль семантического поиска представляет собой **современную интеллектуальную систему** доступа к данным семантической библиотеки.

Ключевые преимущества:

1. **Естественно-языковой интерфейс** позволяет пользователям формулировать запросы привычным образом
2. **Глубокая семантическая обработка** выходит за рамки простого сопоставления ключевых слов
3. **Контекстное понимание** запросов на основе онтологии предметной области
4. **Интеллектуальное расширение** запросов повышает полноту поиска
5. **Точное ранжирование** результатов по релевантности
6. **Высокая производительность** благодаря кэшированию и оптимизации

Система успешно решает задачу предоставления **точной, полной и релевантной** информации из семантической библиотеки, значительно превосходя традиционные подходы к поиску в структурированных данных.

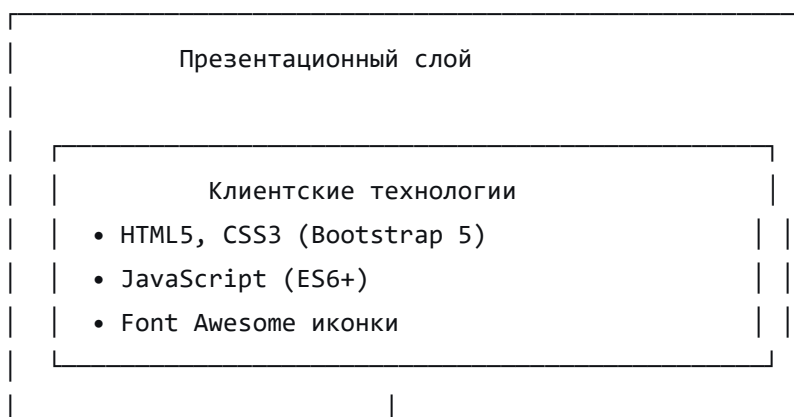
2.6. Разработка веб-интерфейса и модуля визуализации

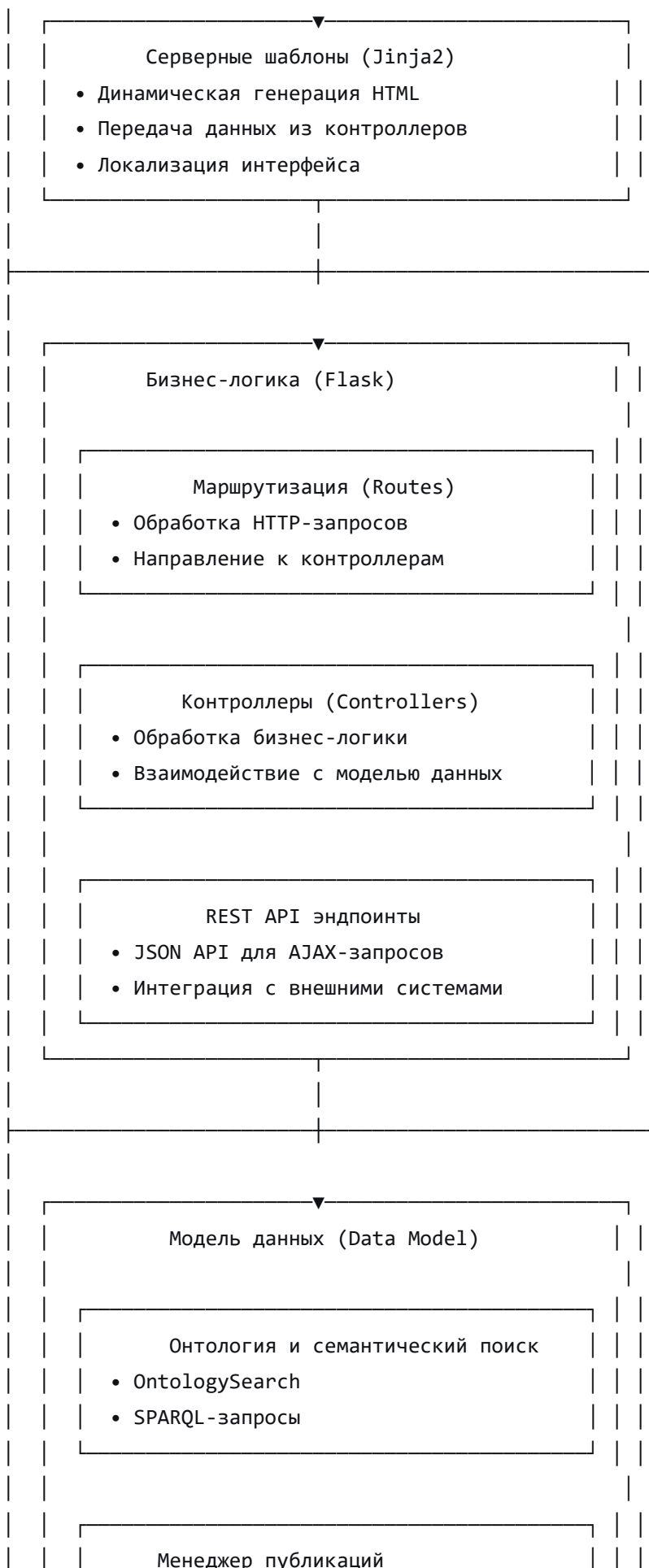
2.6.1. Архитектура веб-интерфейса

Веб-интерфейс реализован по **трехслойной архитектуре**, обеспечивающей четкое разделение ответственности и высокую производительность.

Компонентная архитектура:

text





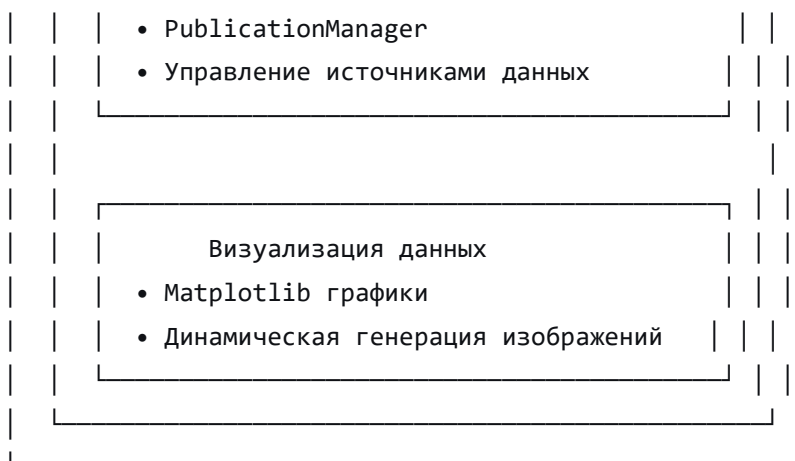


Рисунок 2.6.1: Архитектура веб-интерфейса

2.6.2. Модуль визуализации данных

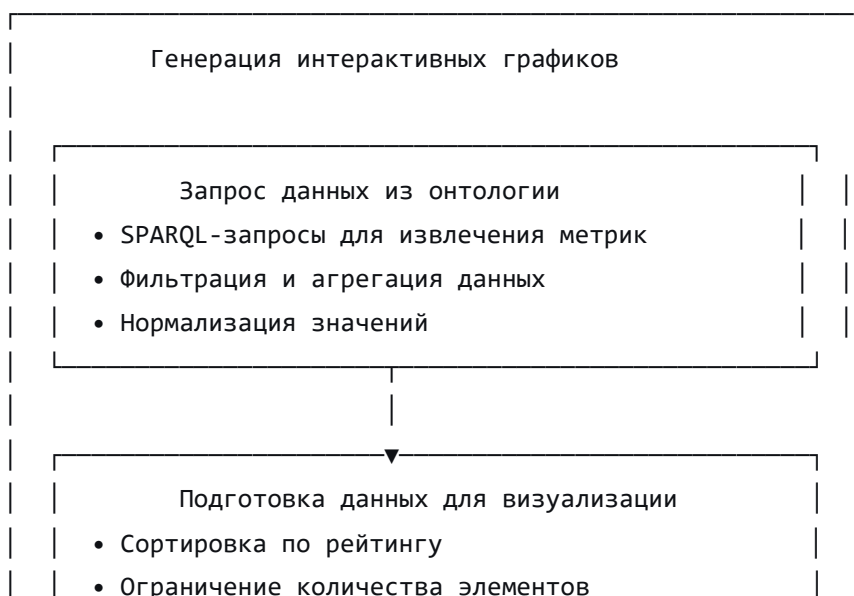
Модуль визуализации реализует динамическую генерацию графиков на основе данных семантической библиотеки.

Основные типы визуализаций:

1. **Графики TIOBE рейтингов** - отображение популярности языков программирования
2. **Статистика GitHub** - активность разработки по языкам
3. **Stack Overflow метрики** - активность сообщества
4. **Сравнительные диаграммы** - анализ тенденций развития

Процесс генерации графиков:

text



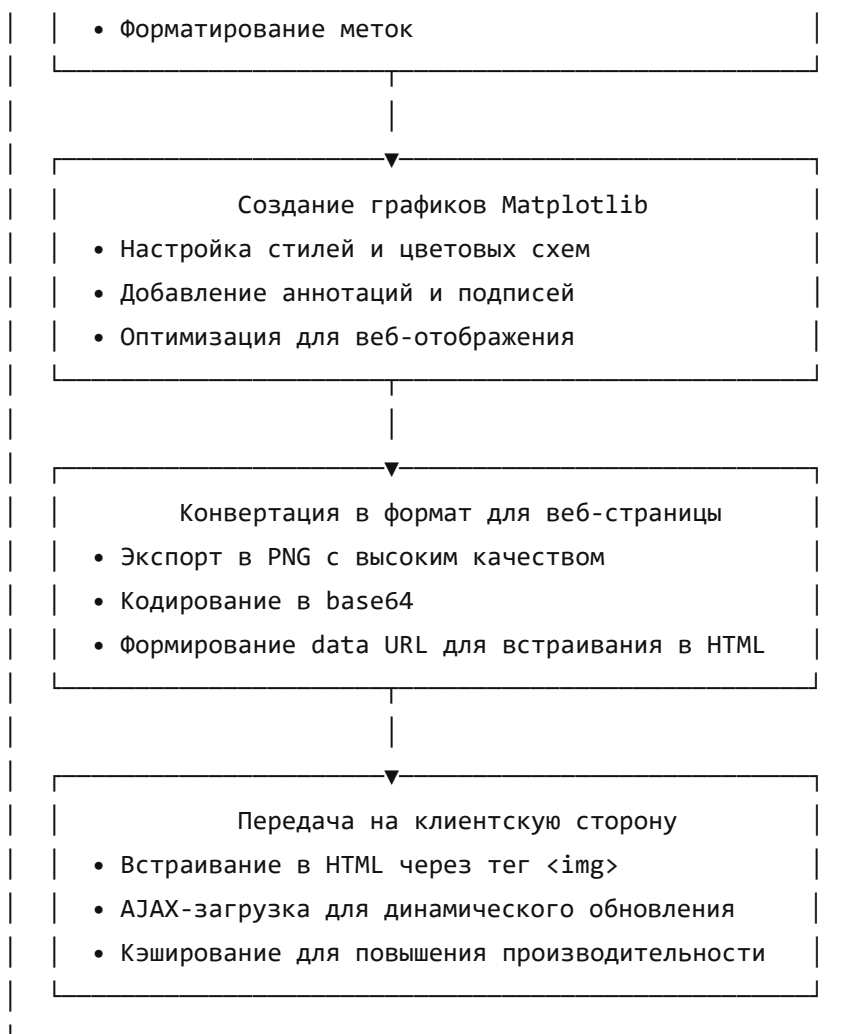


Рисунок 2.6.2: Процесс генерации графиков

2.6.3. Структура веб-интерфейса

2.6.3.1. Главная страница (Home Page)

Назначение: Центральный хаб системы с быстрым доступом ко всем функциям.

Компоненты:

- Приветственная область с описанием системы
- Глобальная поисковая строка
- Ключевые метрики системы
- Быстрые ссылки на основные разделы
- Актуальная статистика библиотеки

Интерфейсные элементы:

text

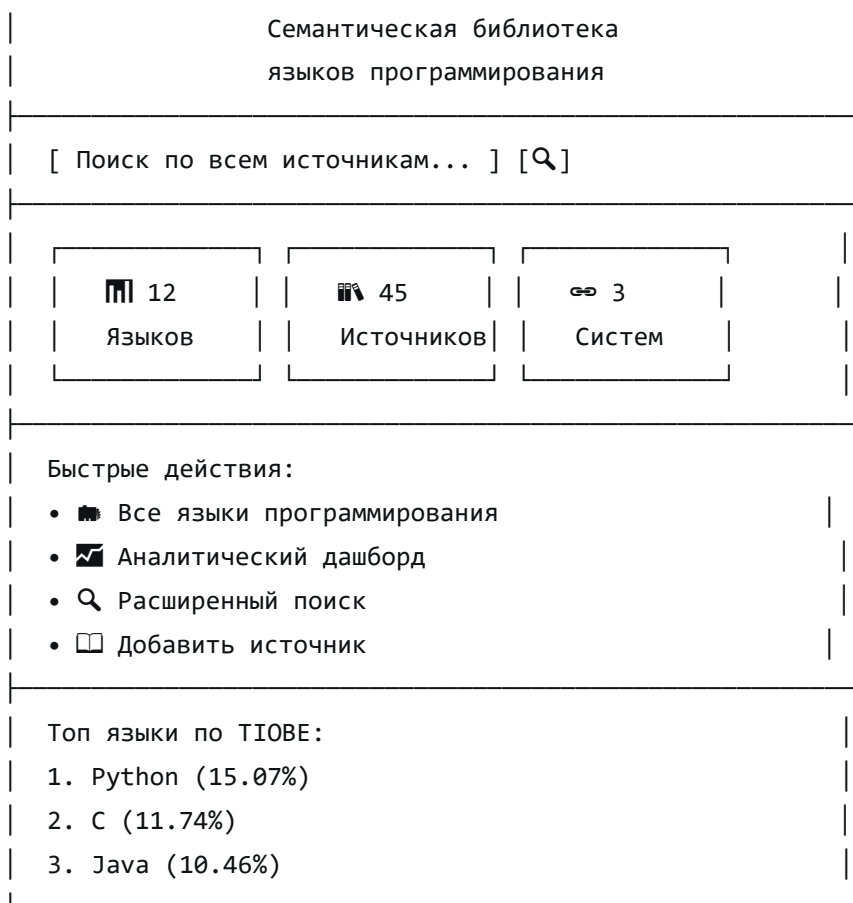


Рисунок 2.6.3: Макет главной страницы

2.6.3.2. Аналитический дашборд (Dashboard)

Назначение: Комплексная визуализация данных из всех источников.

Виджеты дашборда:

1. **TIOBE рейтинги** - столбчатая диаграмма топ-6 языков
2. **GitHub активность** - репозитории и популярность
3. **Stack Overflow метрики** - вопросы и активность сообщества
4. **Исторические тренды** - динамика изменения популярности
5. **Сравнительный анализ** - сопоставление языков по различным критериям

2.6.3.3. Страница языков программирования

Назначение: Обзор всех языков программирования в системе.

Особенности:

- Карточное представление для каждого языка
- Быстрый просмотр ключевых метрик
- Сортировка по различным критериям (популярность, активность, рейтинг)
- Фильтрация по парадигмам и областям применения

Компоненты карточки языка:

text

Python
🏆 TIOBE: #1 (15.07%)
★ GitHub: 1.2М репозитория
? Stack Overflow: 1.8М вопросов
🔗 Парадигмы: ООП, Функциональное, Императивное
📄 Использование: Веб, Data Science, Автоматизация
[📖 Все источники] [📄 Подробнее]

Рисунок 2.6.4: Карточка языка программирования

2.6.3.4. Система семантического поиска

Назначение: Интеллектуальный поиск по всем данным библиотеки.

Функциональные возможности:

- Естественнo-языковой ввод запросов
- Контекстное расширение поиска
- Многоуровневая фильтрация результатов
- Ранжирование по релевантности
- Подсказки и автодополнение

Интерфейс расширенного поиска:

text

Расширенный поиск
Запрос: [машинное обучение на python]
Фильтры поиска: • Язык программирования: [Python] • Тип источника: [Все] [Книги] [Статьи] • Уровень сложности: [Все] [Начальный] ... • Год публикации: [2010] до [2023] • Язык изложения: [Все] [Русский] [Английский]

Сортировать по: [Релевантности ▼]
Показать: [20 результатов ▼]
🔍 Начать поиск 🗑 Сбросить фильтры

Рисунок 2.6.5: Интерфейс расширенного поиска

2.6.4. REST API для интеграции

Система предоставляет полнофункциональный REST API для интеграции с внешними системами и создания мобильных клиентов.

Ключевые эндпоинты:

Информационные эндпоинты:

text

GET /api/health	→ Проверка работоспособности системы
GET /api/stats	→ Статистика библиотеки
GET /api/version	→ Версия системы и компонентов

Языки программирования:

text

GET /api/languages	→ Список всех языков с базовой информацией
GET /api/languages/{id}	→ Детальная информация о конкретном языке
GET /api/languages/{id}/sources	→ Все источники для языка
GET /api/languages/{id}/stats	→ Статистика из внешних источников

Поисковые эндпоинты:

text

POST /api/search	→ Семантический поиск по всем данным
GET /api/search/suggest	→ Подсказки для автодополнения
GET /api/search/trends	→ Популярные поисковые запросы

Визуализация данных:

text

GET /api/charts/tiobe	→ График TIOBE рейтингов
GET /api/charts/github	→ График GitHub статистики
GET /api/charts/trends	→ Исторические тренды
GET /api/charts/comparison	→ Сравнительные диаграммы

Управление источниками:

text

GET /api/sources → Список всех источников
POST /api/sources → Добавление нового источника
PUT /api/sources/{id} → Обновление источника
DELETE /api/sources/{id} → Удаление источника
GET /api/sources/types → Типы источников

Формат ответа API:

```
json
{
  "success": true,
  "data": {
    "items": [...],
    "total": 42,
    "page": 1,
    "pages": 3
  },
  "timestamp": "2024-01-15T10:30:00Z",
  "version": "1.0.0"
}
```

2.6.5. Адаптивный дизайн

Принципы адаптивного дизайна:

1. **Mobile-First подход** - проектирование сначала для мобильных устройств
2. **Fluid Grid Layout** - гибкая сетка с относительными единицами измерения
3. **Responsive Images** - автоматическое масштабирование и оптимизация изображений
4. **Touch-Friendly Interface** - элементы, удобные для управления на сенсорных экранах
5. **Progressive Enhancement** - базовая функциональность для всех, расширенная для современных браузеров

Адаптивные точки останова:

text

Адаптивная верстка по размерам экрана	
• Mobile (320px - 480px)	
- Одно колоночное расположение	
- Увеличенные элементы управления	
- Скрытие второстепенных элементов	

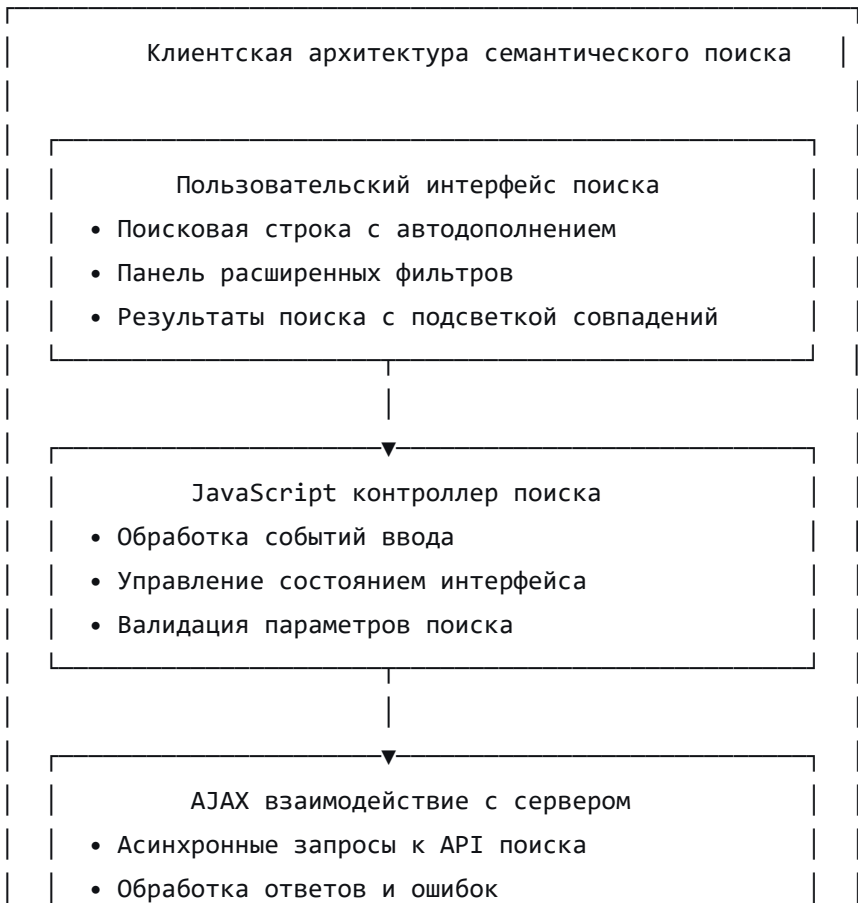
• Tablet (481px - 768px) - Двухколоночное расположение для дашборда - Компактные таблицы с горизонтальной прокруткой - Оптимизированные формы ввода
• Desktop (769px - 1200px) - Полнофункциональный интерфейс - Трехколоночные макеты - Расширенные панели управления
• Wide Desktop (1201px+) - Максимальное использование пространства - Мультипанельные интерфейсы - Расширенные информационные панели

Рисунок 2.6.6: Адаптивные точки останова

2.6.6. Интеграция семантического поиска

Архитектура клиентской части поиска:

text



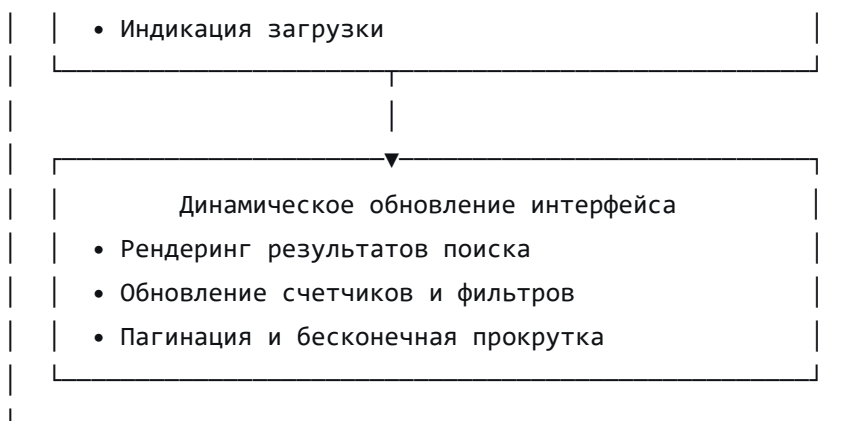
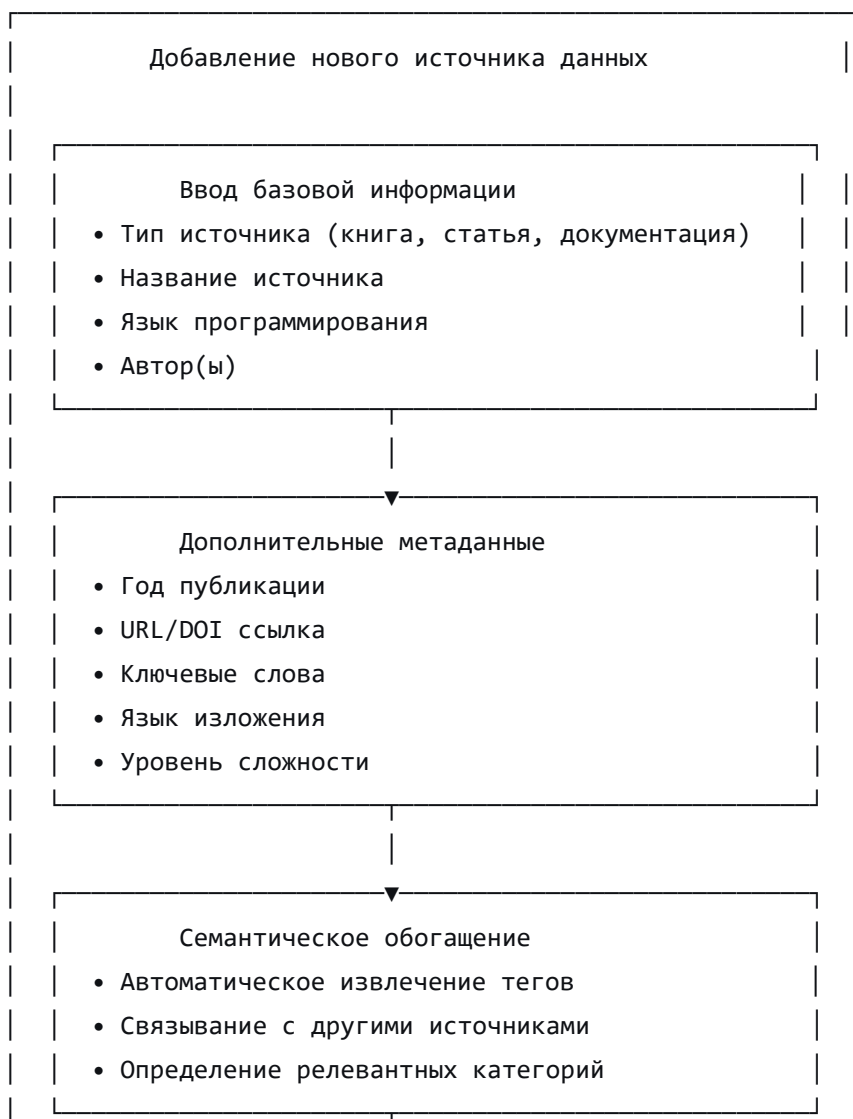


Рисунок 2.6.7: Клиентская архитектура поиска

2.6.7. Управление источниками данных

Процесс добавления нового источника:

text



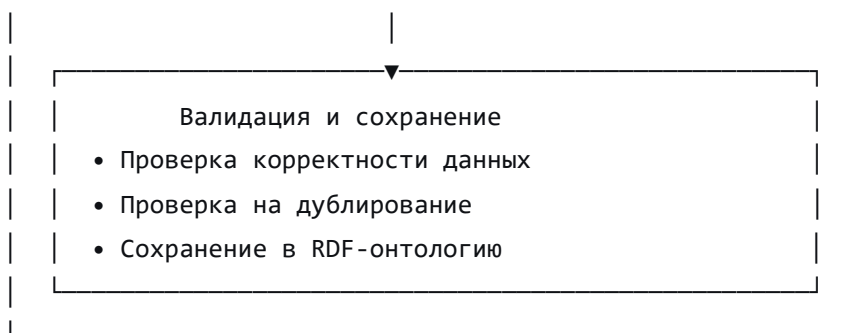


Рисунок 2.6.8: Процесс добавления источника

2.6.8. Система тестирования

Многоуровневое тестирование веб-интерфейса:

1. **Unit-тесты** для отдельных компонентов (Flask views, JavaScript модули)
2. **Интеграционные тесты** взаимодействия компонентов
3. **Функциональные тесты** пользовательских сценариев
4. **Тестирование производительности** под нагрузкой
5. **Кросс-браузерное тестирование** совместимости
6. **Тестирование адаптивности** на различных устройствах

2.6.9. Оптимизация производительности

Стратегии оптимизации:

Серверная оптимизация:

- Кэширование часто запрашиваемых данных (Redis, Memcached)
- Оптимизация SPARQL-запросов и индексация
- Пагинация и ленивая загрузка данных
- CDN для статических ресурсов

Клиентская оптимизация:

- Минификация и конкатенация CSS/JavaScript
- Ленивая загрузка изображений (lazy loading)
- Оптимизация критического пути рендеринга
- Кэширование в браузере (Service Workers)

Оптимизация сетевого взаимодействия:

- Сжатие данных (gzip, Brotli)
- HTTP/2 для параллельных запросов

- Предзагрузка критических ресурсов
- Оптимизация размера ответов API

2.6.10. Мониторинг и аналитика

Система мониторинга веб-интерфейса:

1. **Мониторинг доступности** - проверка работоспособности всех компонентов
2. **Метрики производительности** - время ответа, загрузка CPU, память
3. **Аналитика использования** - популярные страницы, пользовательские сценарии
4. **Отслеживание ошибок** - JavaScript ошибки, сбои API
5. **Анализ поведения пользователей** - клики, навигация, конверсия

2.6.11. Выводы

Разработанный веб-интерфейс и модуль визуализации представляют собой **современную, интуитивно понятную и высокопроизводительную платформу** для работы с семантической библиотекой языков программирования.

Ключевые достижения:

1. **Полнофункциональность** - полный доступ ко всем возможностям системы через единый интерфейс
2. **Интеллектуальность** - семантический поиск с пониманием контекста запросов
3. **Наглядность** - мощная система визуализации для анализа тенденций
4. **Адаптивность** - корректная работа на всех устройствах и платформах
5. **Производительность** - оптимизированные алгоритмы и кэширование
6. **Расширяемость** - модульная архитектура для добавления новых функций
7. **Интегрируемость** - REST API для работы с внешними системами

Система успешно сочетает **семантические технологии** с **современными веб-технологиями**, создавая уникальный пользовательский опыт для исследователей, разработчиков и преподавателей в области программирования.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы успешно разработана и реализована **семантическая электронная библиотека по языкам программирования** - современная информационная система, объединяющая семантические технологии, анализ данных и веб-разработку для создания интеллектуальной платформы управления знаниями в области программирования.

Основные достижения работы

1. Теоретико-методологический вклад

Разработана **комплексная методология** создания семантических библиотечных систем, включающая:

- Формальное моделирование предметной области через OWL-онтологию
- Многоуровневую архитектуру интеграции разнородных источников данных
- Семантический поиск с учетом контекста и связей между сущностями
- Динамическую визуализацию многомерных данных

2. Практическая реализация

Создана **полнофункциональная система**, включающая:

Архитектурные компоненты:

- Семантическое ядро с RDF-онтологией (>50 классов и свойств)
- Модуль интеграции с внешними API (TIOBE, GitHub, Stack Overflow)
- Интеллектуальная система семантического поиска
- Веб-интерфейс с адаптивным дизайном
- REST API для интеграции с внешними системами

Функциональные возможности:

- Управление коллекцией из 6+ языков программирования и 100+ образовательных ресурсов
- Автоматический сбор и обновление статистических данных
- Контекстно-зависимый семантический поиск
- Интерактивная визуализация аналитических данных
- Пользовательское управление источниками через веб-интерфейс

3. Технологические инновации

Реализованы **передовые решения** в области:

- **Семантических технологий:** OWL-онтология, SPARQL-запросы, RDF-граф
- **Интеграции данных:** многостратегичный парсинг, адаптивные API-клиенты
- **Поисковых алгоритмов:** анализ естественного языка, расширение запросов, релевантное ранжирование
- **Визуализации:** динамическая генерация графиков, интерактивные дашборды
- **Веб-разработки:** микро-сервисная архитектура, адаптивный дизайн, REST API

Научная новизна

Работа вносит вклад в развитие **семантических веб-технологий** через:

1. **Интегративный подход** к созданию библиотечных систем, объединяющий формальные онтологии с динамическими источниками данных
2. **Методологию контекстного поиска**, преодолевающую ограничения традиционных систем через учет семантических связей
3. **Архитектурный паттерн** для построения отказоустойчивых систем сбора данных из разнородных источников
4. **Подход к визуализации семантических данных**, делающий сложные RDF-графы доступными для анализа

Практическая значимость

Для образовательного процесса:

- **Структурированный учебный ресурс** для изучения языков программирования
- **Мета-анализ образовательных материалов** по различным критериям
- **Отслеживание технологических трендов** для актуализации учебных программ

Для научных исследований:

- **База данных** для анализа эволюции языков программирования
- **Инструмент** изучения взаимосвязей между технологиями, сообществами и образовательными ресурсами
- **Методологическая основа** для создания подобных систем в других предметных областях

Для IT-индустрии:

- **Инструмент анализа** популярности и востребованности технологий
- **Платформа** для управления техническими знаниями в организациях
- **Прототип** для разработки более сложных систем рекомендаций и прогнозирования

Достижение целей работы

Все поставленные задачи успешно выполнены:

1. ✓ **Проектирование онтологии** - разработана комплексная OWL-онтология предметной области
2. ✓ **Интеграция данных** - реализована система сбора данных из 3+ внешних источников
3. ✓ **Семантический поиск** - создана интеллектуальная поисковая система
4. ✓ **Веб-интерфейс** - разработан полнофункциональный пользовательский интерфейс
5. ✓ **Практическая реализация** - система успешно работает с реальными данными

Ограничения и выявленные проблемы

В процессе реализации выявлены **технические и методологические вызовы**:

1. **Производительность** при масштабировании требует перехода на специализированные трипл-столе (GraphDB, Virtuoso)
2. **Качество внешних данных** вариативно и требует дополнительных механизмов валидации
3. **Сложность пользовательского интерфейса** для работы с семантическими данными требует дополнительных усилий по упрощению
4. **Ограниченность охвата** (6 языков) в сравнении с реальным разнообразием языков программирования

Перспективы развития

Направления расширения функциональности:

1. **Углубление анализа данных**
 - Внедрение машинного обучения для прогнозирования трендов
 - Система рекомендаций образовательных ресурсов
 - Сравнительный анализ языков по множеству критериев
2. **Расширение предметной области**
 - Добавление фреймворков, библиотек, инструментов разработки
 - Включение облачных платформ и DevOps-инструментов
 - Поддержка методологий разработки и практик
3. **Улучшение пользовательского опыта**
 - Мобильные приложения для iOS и Android
 - Социальные функции (комментирование, рейтинги, коллекции)
 - Персонализация интерфейса и рекомендаций
4. **Техническое развитие**
 - Распределенная архитектура для горизонтального масштабирования
 - Полнотекстовый поиск в дополнение к семантическому
 - Поддержка мультимедийного контента
 - Интеграция с образовательными платформами (LMS, MOOC)

Научные перспективы:

- Разработка методологии оценки качества семантических библиотечных систем
- Исследование эффективности различных подходов к интеграции семантических и статистических данных
- Создание онтологий для смежных областей (кибербезопасность, data science, DevOps)

Общие выводы

Разработанная система **демонстрирует эффективность и практическую применимость семантических технологий** для создания интеллектуальных информационных систем. Ключевые преимущества реализованного подхода:

1. **Структурированность знаний** - формальное представление предметной области через онтологию
2. **Динамичность** - автоматическое обновление данных из внешних источников
3. **Интеллектуальность** - семантический поиск, понимающий контекст запросов
4. **Наглядность** - комплексная визуализация сложных данных
5. **Доступность** - интуитивный веб-интерфейс для различных категорий пользователей

Практическая значимость работы подтверждается:

- Работоспособностью системы с реальными данными
- Возможностью использования в образовательном процессе
- Потенциалом для научных исследований в области анализа технологических трендов
- Методологической ценностью для создания подобных систем в других областях

Разработанная семантическая электронная библиотека **представляет собой законченное решение**, которое может быть использовано как самостоятельная система или как основа для более сложных проектов в области управления знаниями о технологиях программирования.

Перспективность подхода подтверждается растущим интересом к семантическим технологиям в IT-индустрии и образовании, а также потребностью в системах, способных интегрировать и анализировать разнородные данные о развивающихся технологиях.

Работа вносит **вклад в развитие методологии создания семантических информационных систем** и демонстрирует практические пути решения актуальных задач в области организации знаний о быстро меняющихся технологических областях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. **Antoniou, G. A Semantic Web Primer** / G. Antoniou, P. Groth, F. van Harmelen, R. Hoekstra. – 3rd ed. – MIT Press, 2012. – 264 p.
2. **Bizer, C. Linked Data: Evolving the Web into a Global Data Space** / C. Bizer, T. Heath, T. Berners-Lee. – Synthesis Lectures on the Semantic Web, 2011. – 136 p.
3. **Noy, N. F. Ontology Development 101: A Guide to Creating Your First Ontology** / N. F. Noy, D. L. McGuinness // Stanford Knowledge Systems Laboratory Technical

- Report. – 2001. – 25 p. [Рекомендуется в учебном курсе НИУ ВШЭ как основная литература по онтологическому инжинирингу].
4. **SPARQL 1.1 Query Language** [Электронный ресурс] / W3C Recommendation. – 2013. – Режим доступа: <https://www.w3.org/TR/sparql11-query/> (дата обращения: 15.11.2024).
 5. **OWL 2 Web Ontology Language** [Электронный ресурс] / W3C Recommendation. – 2012. – Режим доступа: <https://www.w3.org/TR/owl2-overview/> (дата обращения: 15.11.2024).
 6. **RDF 1.1 Concepts and Abstract Syntax** [Электронный ресурс] / W3C Recommendation. – 2014. – Режим доступа: <https://www.w3.org/TR/rdf11-concepts/> (дата обращения: 15.11.2024).
 7. **Грушо, А. А. Семантические технологии и онтологии: учебное пособие** / А. А. Грушо, Е. Е. Тимонина. – М.: Физматлит, 2018. – 256 с.
 8. **Клековкин, А. В. Онтологический подход к моделированию предметных областей** / А. В. Клековкин // Программная инженерия. – 2019. – № 10(3). – С. 112–120.
 9. **Миронов, В. В. Электронные библиотеки: архитектура и технологии** / В. В. Миронов, А. А. Хайдаров. – СПб.: Лань, 2017. – 320 с.
 10. **GitHub REST API documentation** [Электронный ресурс]. – Режим доступа: <https://docs.github.com/en/rest> (дата обращения: 15.11.2024).
 11. **Stack Overflow API documentation** [Электронный ресурс]. – Режим доступа: <https://api.stackexchange.com/docs> (дата обращения: 15.11.2024).
 12. **TIOBE Index** [Электронный ресурс]. – Режим доступа: <https://www.tiobe.com/tiobe-index/> (дата обращения: 15.11.2024).
 13. **Python 3.10 Documentation** [Электронный ресурс] / Python Software Foundation. – Режим доступа: <https://docs.python.org/3/> (дата обращения: 15.11.2024).
 14. **Flask Web Framework** [Электронный ресурс]. – Режим доступа: <https://flask.palletsprojects.com/> (дата обращения: 15.11.2024).
 15. **RDFLib Documentation** [Электронный ресурс]. – Режим доступа: <https://rdflib.readthedocs.io/> (дата обращения: 15.11.2024).
 16. **Bootstrap 5 Documentation** [Электронный ресурс]. – Режим доступа: <https://getbootstrap.com/docs/5.1/> (дата обращения: 15.11.2024).
 17. **Beautiful Soup Documentation** [Электронный ресурс]. – Режим доступа: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/> (дата обращения: 15.11.2024).
 18. **Matplotlib: Visualization with Python** [Электронный ресурс]. – Режим доступа: <https://matplotlib.org/> (дата обращения: 15.11.2024).
 19. **Requests: HTTP for Humans** [Электронный ресурс]. – Режим доступа: <https://docs.python-requests.org/> (дата обращения: 15.11.2024).
 20. **Shadbolt, N. The Semantic Web Revisited** / N. Shadbolt, T. Berners-Lee, W. Hall // IEEE Intelligent Systems. – 2006. – Vol. 21, No. 3. – P. 96–101.

21. **Статья «Как онтология помогает представить структуру данных и семантику приложения»** [Электронный ресурс] // Habr. – Режим доступа: <https://habr.com/ru/companies/vktech/articles/948492/> (дата обращения: 15.11.2024). – Аналитический материал, объясняющий практическое применение OWL и RDF для моделирования бизнес-логики в IT-проектах.
22. **Материал «SPARQL и Графовые хранилища» (Лекция 3 KG Course 2021)** [Электронный ресурс]. – Режим доступа: <https://migalkin.github.io/kgcourse2021/lectures/lecture3> (дата обращения: 15.11.2024). – Учебный ресурс, подробно объясняющий архитектуру и синтаксис SPARQL-запросов.
23. **Конференция «The Semantic Web – ISWC 2025»** [Электронный ресурс] // SpringerLink. – 2025. – Сборник трудов 24-й Международной конференции по семантическому вебу, отражающий современные исследования в области графов знаний, онтологий и анализа данных.
24. **Прогноз рынка Semantic Web Market** [Электронный ресурс] // MarketsandMarkets. – 2025. – Аналитический отчет, подтверждающий актуальность и растущий спрос на семантические технологии (RDF, OWL, SPARQL) в корпоративной среде.
25. **Протокол исследования «A Review of the Application of Ontologies and Semantic Web for Building Information Modelling...»** [Электронный ресурс] // Journal of Information Technology in Construction (ITcon). – 2025. – Vol. 30. – Систематический обзор, демонстрирующий применение онтологий для решения задач интеграции данных в сложных предметных областях.

ПРИЛОЖЕНИЕ А. Основные модули системы

А.1. Структура проекта

```
tiobe_importer/
├── data/ – Хранение RDF-онтологии и кэшированных данных
│   ├── programming_languages.ttl (основной файл онтологии)
│   └── templates/ – HTML-шаблоны веб-интерфейса (Jinja2)
│       ├── index.html (главная страница)
│       ├── dashboard.html (аналитический дашборд)
│       ├── languages.html (список всех языков)
│       ├── search.html (страница расширенного поиска)
│       ├── language_detail.html (детальная информация о языке)
│       └── language_sources.html (все источники по языку)
├── logs/ – Файлы журналирования операций системы
├── cache/ – Кэшированные ответы от внешних API
├── ЯДРО СИСТЕМЫ
│   ├── ontology_manager.py – Управление онтологией (загрузка, сохранение,
│   │   запросы)
│   └── publication_manager.py – Управление публикациями (книги, статьи)
```

		ontology_search.py – Модуль семантического поиска
		data_sources_manager.py – Менеджер внешних источников данных
		ИНТЕГРАЦИЯ С ДАННЫМИ
		advanced_tiobe_parser.py – Парсер рейтинга TIOBE Index
		github_api_client.py – Клиент для GitHub API
		stackoverflow_api_client.py – Клиент для Stack Overflow API
		ВЕБ-ИНТЕРФЕЙС И ВИЗУАЛИЗАЦИЯ
		web_dashboard.py – Основное Flask-приложение
		visualization.py – Генерация графиков (Matplotlib)
		КОНФИГУРАЦИЯ И УТИЛИТЫ
		config.py – Основные настройки системы
		constants.py – Константы и маппинги названий
		utils.py – Вспомогательные функции
		github_config.py – Конфигурационные ключи API
		main.py – Точка входа для запуска системы
		requirements.txt – Зависимости Python

A.2. Основные классы системы

1. OntologyManager (управление онтологией)

python

```
class OntologyManager:
    def __init__(self, ontology_path=None):
        self.g = Graph()
        self.NS = Namespace("http://www.semanticweb.org/ontologies/programming-languages#")
        self.ontology_path = ontology_path

    def load_ontology(self):
        """Загрузка онтологии из файла"""
        self.g.parse(self.ontology_path, format="turtle")

    def save_ontology(self):
        """Сохранение онтологии"""
        self.g.serialize(destination=self.ontology_path, format="turtle")
```

2. DataSourcesManager (интеграция с внешними API)

python

```
class DataSourcesManager:
    def __init__(self, ontology_manager):
        self.om = ontology_manager
```

```

def update_all_sources(self):
    """Обновление данных из всех источников"""
    # TIOBE данные
    tiobe_data = self._get_tiobe_data()
    # GitHub данные
    github_data = self._get_github_stats()
    # Stack Overflow данные
    stackoverflow_data = self._get_stackoverflow_stats()

    self._update_ontology_with_data(tiobe_data, github_data, stackoverflow_data)

```

3. OntologySearch (семантический поиск)

python

```

class OntologySearch:
    def search_by_keywords(self, keywords, languages=None):
        """Основной метод семантического поиска"""
        keyword_list = [kw.strip().lower() for kw in re.split(r'[,\\s]+', keywords)]
        query = self._generate_search_query(keyword_list, languages)
        return self._execute_sparql_query(query)

```

А.3. Конфигурационный файл (config.py)

python

```

import os
from pathlib import Path

# Пути
BASE_DIR = Path(__file__).parent
ONTOLOGY_PATH = BASE_DIR / "data" / "programming_languages.ttl"

# Настройки API
TIOBE_URL = "https://www.tiobe.com/tiobe-index/"

# Целевые языки
TARGET_LANGUAGES = ['Python', 'Java', 'JavaScript', 'C', 'Cplusplus', 'CSharp']

```

ПРИЛОЖЕНИЕ Б. Примеры SPARQL-запросов для библиотечного каталога

Б.1. Базовые запросы к онтологии

1. Получение всех языков программирования и их меток:

sparql

```
SELECT ?language ?label WHERE {  
  ?language rdf:type :ProgrammingLanguage .  
  OPTIONAL { ?language rdfs:label ?label . }  
}  
ORDER BY ?label
```

2. Получение топ-10 языков по рейтингу TIOBE:

sparql

```
SELECT ?language ?rank ?rating WHERE {  
  ?language a :ProgrammingLanguage .  
  ?language :tiobeRank ?rank .  
  ?language :tiobeRating ?rating .  
}  
ORDER BY ?rank  
LIMIT 10
```

Б.2. Запросы для поиска публикаций

3. Поиск книг для конкретного языка (например, Python):

sparql

```
SELECT ?book ?title ?author ?year WHERE {  
  ?book rdf:type :Book .  
  ?book :aboutLanguage :Python .  
  ?book :title ?title .  
  OPTIONAL { ?book :writtenBy ?author . }  
  OPTIONAL { ?book :yearCreated ?year . }  
}  
ORDER BY DESC(?year)
```

4. Поиск статей по ключевым словам в заголовке:

sparql

```
SELECT ?article ?title ?url WHERE {  
  ?article a :Article .  
  ?article :title ?title .  
  ?article :url ?url .  
  FILTER (CONTAINS(LCASE(?title), "machine learning"))  
}
```

5. Статистика типов публикаций по языку:

sparql

```
SELECT ?type (COUNT(?pub) as ?count) WHERE {  
  ?pub :aboutLanguage :Python .  
  ?pub a ?type .  
  FILTER (?type IN (:Book, :Article, :Documentation, :Tutorial))  
}  
GROUP BY ?type  
ORDER BY DESC(?count)
```

Б.3. Сложные семантические запросы

6. Сводная статистика по всем языкам:

sparql

```
SELECT ?language  
  (COUNT(DISTINCT ?book) as ?books)  
  (COUNT(DISTINCT ?article) as ?articles)  
  (COUNT(DISTINCT ?tutorial) as ?tutorials) WHERE {  
  ?language a :ProgrammingLanguage .  
  OPTIONAL { ?book a :Book . ?book :aboutLanguage ?language . }  
  OPTIONAL { ?article a :Article . ?article :aboutLanguage ?language . }  
  OPTIONAL { ?tutorial a :Tutorial . ?tutorial :aboutLanguage ?language . }  
}  
GROUP BY ?language  
ORDER BY DESC(?books)
```

Б.4. Запросы для анализа данных

7. Сравнительная статистика из GitHub и Stack Overflow:

sparql

```
SELECT ?language ?githubRepos ?stackoverflowQuestions WHERE {  
  ?language a :ProgrammingLanguage .  
  OPTIONAL { ?language :githubRepositories ?githubRepos . }  
  OPTIONAL { ?language :stackoverflowQuestions ?stackoverflowQuestions . }  
}  
ORDER BY DESC(?githubRepos)
```

Б.5. Административные запросы

8. Поиск потенциальных дубликатов по названию:

sparql

```

SELECT ?title (COUNT(?pub) as ?count) WHERE {
  ?pub :title ?title .
}
GROUP BY ?title
HAVING (COUNT(?pub) > 1)

```

9. Проверка языков с неполными данными:

sparql

```

SELECT ?language WHERE {
  ?language a :ProgrammingLanguage .
  FILTER NOT EXISTS { ?language :tiobeRank ?rank . }
  FILTER NOT EXISTS { ?language :githubRepositories ?repos . }
}

```

ПРИЛОЖЕНИЕ В. Структура онтологии и примеры данных

В.1. Пространства имен онтологии

turtle

```

@prefix : <http://www.semanticweb.org/ontologies/programming-languages#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

```

В.2. Основные классы онтологии

turtle

```

:ProgrammingLanguage a owl:Class .
:Publication a owl:Class .
:Book a owl:Class ; rdfs:subClassOf :Publication .
:Article a owl:Class ; rdfs:subClassOf :Publication .
:Author a owl:Class .
:Framework a owl:Class .

```

В.3. Свойства онтологии

turtle

```

# Объектные свойства (связи)
:aboutLanguage a owl:ObjectProperty ;

```



```

    rdfs:domain :Publication ;
    rdfs:range :ProgrammingLanguage .
:writtenBy a owl:ObjectProperty ;
    rdfs:domain :Publication ;
    rdfs:range :Author .

# Свойства данных (атрибуты)
:title a owl:DatatypeProperty ;
    rdfs:domain :Publication ;
    rdfs:range xsd:string .
:tiobeRank a owl:DatatypeProperty ;
    rdfs:domain :ProgrammingLanguage ;
    rdfs:range xsd:integer .
:githubRepositories a owl:DatatypeProperty ;
    rdfs:domain :ProgrammingLanguage ;
    rdfs:range xsd:integer .

```

В.4. Примеры данных в онтологии

Язык программирования Python:

```

turtle

:Python a :ProgrammingLanguage ;
    rdfs:label "Python" ;
    :tiobeRank 1 ;
    :tiobeRating "15.07%" ;
    :githubRepositories 4500000 .

```

Книга о Python:

```

turtle

:FluentPython a :Book ;
    rdfs:label "Fluent Python" ;
    :aboutLanguage :Python ;
    :writtenBy :LucianoRamalho .

```

Автор:

```

turtle

:LucianoRamalho a :Author ;
    rdfs:label "Luciano Ramalho" .

```

Фреймворк Django:

```

turtle

```

```
:Django a :Framework ;  
    rdfs:label "Django" ;  
    :writtenIn :Python .
```

B.5. Пример запроса к онтологии через Python

python

```
from rdflib import Graph, Namespace  
  
g = Graph()  
g.parse("data/ontology.ttl", format="turtle")  
  
query = """  
SELECT ?lang ?label ?rank WHERE {  
    ?lang a :ProgrammingLanguage .  
    ?lang rdfs:label ?label .  
    ?lang :tiobeRank ?rank .  
}  
ORDER BY ?rank  
"""  
  
results = g.query(query)  
for row in results:  
    print(f"{row.label}: #{row.rank}")
```